COMSATS University Islamabad, Sahiwal Campus

Grow Mart

By

Hassan Naveed **CIIT/FA22-BSE-030/SWL**
Muhammad Sadam **CIIT/FA22-BSE-033/SWL**
Raja Noshairwan Kamal **CIIT/FA22-BSE-039/SWL**

Supervisor

Mr. Shehzad Ali

Bachelor of Science in Computer Science (2022-2026)

The candidates confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

COMSATS University Islamabad, Sahiwal Campus

Grow Mart

A project presented to
COMSATS Institute of Information Technology, Islamabad

Bachelor of Science in Computer Science (2021-2025)

By

Hassan Naveed	**CIIT/FA22-BSE-030/SWL**
Muhammad Sadam	**CIIT/FA22-BSE-033/SWL**
Raja Noshairwan Kamal	**CIIT/FA22-BSE-039/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Hassan Naveed

Muhammad Sadam

Raja Noshairwan Kamal

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) "Grow Mart" was developed by Hassan Naveed (CIIT/FA22-BSE-030/SWL), Muhammad Sadam (CIIT/FA22-BSE-033/SWL), and Raja Noshairwan Kamal (CIIT/FA22-BSE-039/SWL) under the supervision of "Mr. Shehzad Ali" and that in their opinion, it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

Internal Examiner

Head Of Department
(Department of Computer Science)

Executive Summary

Grow Mart is a comprehensive mobile application that we have personally developed to redefine the plant and gardening e-commerce experience by providing a dedicated digital marketplace for plant enthusiasts, gardeners, and small-scale sellers. During our research and observation of user behavior in the plant commerce industry, we discovered that many people struggle to find a centralized platform specifically designed for buying and selling plant-related products such as indoor/outdoor plants, pots, fertilizers, and gardening tools. Grow Mart is our solution to bridge this gap by offering a role-based mobile platform that connects buyers and sellers in a secure, real-time environment.

The platform integrates several core modules: User Registration & Authentication, Product Management, Product Browsing & Search, Shopping Cart Management, Checkout & Payment Processing, Order Management, Review & Rating System, and Notification System. Users can seamlessly register, browse a curated collection of plant-related products, add items to their cart, choose between Stripe online payment or Cash on Delivery (COD), and track their orders in real-time. Sellers can upload products, manage inventory, view analytics, and respond to customer reviews, all within a smooth, interactive interface.

Throughout the development of Grow Mart, we followed the Object-Oriented Methodology using MVVM architecture and the Incremental Model, starting with essential features like authentication and product listing, then expanding the platform with advanced modules like payment integration, push notifications, and seller analytics. This phased approach allowed us to continuously test, refine, and improve the platform based on user feedback and functionality.

This project represents not only our technical capabilities but also our commitment to enhancing user experience and solving real-world problems with technology. Every module, every feature, and every interaction in Grow Mart was built with precision and passion to offer users a modern, convenient, and innovative way to buy and sell plant-related products.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by which we were able to accomplish this challenging task.
We are greatly indebted to our project supervisor, "Mr. Shehzad Ali". Without his personal supervision, advice, and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to him for his encouragement and continual help during this work.
And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Hassan Naveed

Muhammad Sadam

Raja Noshairwan Kamal

Abbreviations

Abbreviation	Meaning
GM	Grow Mart
MVVM	Model-View-ViewModel
FCM	Firebase Cloud Messaging
COD	Cash on Delivery
API	Application Programming Interface
DBMS	Database Management System
HCI	Human-Computer Interaction
OOP	Object-Oriented Programming
NoSQL	Not Only Structured Query Language
JSON	JavaScript Object Notation
CRUD	Create, Read, Update, Delete
UI	User Interface
UX	User Experience
FCM	Firebase Cloud Messaging
IDE	Integrated Development Environment
SRS	Software Requirements Specification
SDD	Software Design Description
UT	Unit Testing
FT	Functional Testing
IT	Integration Testing
UAT	User Acceptance Testing

RBAC	Role-Based Access Control

Table of content

List of Table

List of Figures

Chapter 1

1.1 Introduction

Grow Mart is a comprehensive mobile application we developed to address a common challenge in the plant and gardening e commerce space helping plant enthusiasts and small scale sellers connect through a dedicated, specialized digital marketplace. As e-commerce continues to grow, particularly in niche markets like gardening and plant care, there is a rising demand for digital tools that provide a more focused and personalized shopping experience. Most existing platforms are either too general, lacking plant specific filters and features, or fail to provide adequate buyer and seller communication and order tracking capabilities. Grow Mart bridges this gap by integrating role-based access, real time notifications, secure payment processing, and comprehensive inventory management, all within a user friendly mobile interface.

This platform is built to serve a wide range of users, from casual plant lovers to professional gardeners and small-scale plant sellers. It promotes confident and informed purchasing by enabling users to explore categorized product catalogs, register for personalized services, add items to their cart, choose between online payment or cash on delivery, and track their orders in real-time.

The goal of this project is to enhance user confidence and satisfaction when buying and selling plant-related products online by creating a dedicated, specialized e commerce platform. More than just a technical solution, Grow Mart represents a user-centric approach to solving real-world commerce pain points through innovation and thoughtful design.

1.2 Brief

Grow Mart provides user registration and authentication, product browsing with smart search and filtering, shopping cart management, secure checkout via Stripe or Cash on Delivery, order tracking, review and rating systems, real time push notifications, and a seller analytics dashboard all structured to enhance user confidence and streamline the buying and selling process. Grow Mart combines practical utility with modern technology to deliver a seamless, intelligent, and engaging shopping journey. The system is divided into the following core modules:

1.2.1 User Registration and Profile Management

This module allows users to easily register and manage their accounts. New users can sign up using their email and password via Firebase Authentication, while returning users can log in securely to access personalized features such as order history, saved addresses, and profile settings. Once registered, users can maintain a profile, manage their details, and select their role as either a Buyer or Seller, which determines their access to specific platform features.

1.2.2 Product Management and Smart Search

This module enables sellers to add, update, and delete product listings including plant names, prices, images, descriptions, and categories such as Indoor Plants, Outdoor Plants, Pots, Fertilizers, and Gardening Tools. Buyers can browse through different categories and use smart search filters to narrow down the selection based on product type, price range, or other preferences.

The product catalog displays curated collections complete with images, descriptions, pricing, and specifications. Sellers can upload product images to Firebase Storage, and the system automatically generates image URLs for display in the product catalog.

1.2.3 Shopping Cart and Order Placement

This module allows buyers to manage their plant and gardening product selections efficiently. Users can add products to their cart, adjust quantities, and remove items before proceeding to checkout. The platform supports a simplified and secure order placement system where users can review their items and confirm purchases.

The order process is designed to be user-friendly and transparent, with confirmation notifications and real-time status updates. Sellers can manage orders and update statuses (e.g., Placed → Processing → Shipped → Delivered) from their dedicated dashboard.

1.2.4 Checkout and Payment Processing

This module handles the secure processing of payments through two methods: Stripe for online card payments and Cash on Delivery (COD) for users who prefer to pay upon receipt. The system validates payment information, processes transactions through the Stripe payment gateway, and updates order status on successful payment. All financial transactions are secured using industry-standard encryption protocols to protect user data during payment processes.

1.2.5 Review and Rating System

This module enables buyers to rate and review products after successful delivery. Users can submit star ratings and written feedback, which are then displayed on the product page to help other buyers make informed purchasing decisions. This feature builds community trust and encourages quality standards among sellers.

1.2.6 Notification System

This module handles real-time communication with users through Firebase Cloud Messaging (FCM). Both buyers and sellers receive push notifications for important events such as order confirmations, payment updates, delivery status changes, and new reviews. If a user has disabled notifications, alerts are saved in the in-app notification log for later viewing.

1.2.7 Seller Analytics Dashboard

The seller dashboard provides valuable insights into sales performance, revenue trends, and order statistics. Sellers can view analytics related to their product listings, track which items are performing well, and monitor their overall business growth through the platform. This empowers sellers to make data-driven decisions about their inventory and pricing strategies.

1.3 Relevance to Course Modules

Grow Mart demonstrates the practical application of several core computer science modules covered throughout our Bachelor of Computer Science degree. It reflects how theoretical knowledge can be transformed into a functional mobile e-commerce platform.

1.3.1 Mobile Application Development

The front end of Grow Mart was built using Flutter (Dart) with the MVVM architecture pattern. These technologies, learned during mobile application development courses, enabled us to design a responsive, intuitive, and cross-platform user interface that supports real-time interactions and smooth navigation through plant-related products.

1.3.2 Software Engineering

Principles from software engineering such as modular design, incremental development, and requirement analysis were fundamental in structuring the system. These helped us plan, organize, and manage the development phases of multiple modules including authentication, product management, order processing, and notifications.

1.3.3 Database Management Systems

Using Firebase Firestore a NoSQL cloud-hosted database we designed the backend to handle user profiles, product catalogs, order histories, shopping carts, and review data. This knowledge came directly from DBMS coursework and was applied to optimize data structures for real-time synchronization.

1.3.4 Human-Computer Interaction (HCI)

From interface layout to navigation flow, the design was shaped by HCI principles to ensure users could interact comfortably with the platform. Features such as role-based dashboards, clear icons, readable text, and consistent layouts all reflect usability techniques emphasized in HCI courses.

1.4 Project Background

The concept for Grow Mart arose from observing how plant and gardening e-commerce still lacks a specialized, dedicated platform for this growing niche market. Plant lovers and small-scale gardening businesses lack a centralized platform for trading plant-related products. Current online options are either too general (like OLX or Facebook Marketplace) or lack plant-specific filters, buyer-seller communication, or reliable delivery tracking. This challenge sparked the idea to develop a platform that bridges this gap using modern mobile development technologies.

Grow Mart is a mobile-based platform that enables users to buy and sell plant-related products using their smartphones. The platform includes a catalog of products, smart search and filtering, a real-time notification system, shopping cart, secure payments, and role-specific dashboards. Users can interact with the system in a way that provides a complete e-commerce experience tailored specifically for the plant and gardening community.

The inclusion of role-based access control, real-time notifications, seller analytics, and secure payment integration makes Grow Mart a comprehensive and practical solution for modern e commerce in the plant and gardening sector. It serves as a direct response to the real challenges faced by both plant buyers and sellers.

1.5 Literature Review

Table 1-1 summarizes the gaps in existing online marketplace platforms such as OLX, Daraz, and Facebook Marketplace, highlighting issues such as lack of plant-specific focus, limited buyer-seller communication, no order tracking, and insufficient seller analytics. Grow Mart addresses these gaps by offering an all-in-one solution with a specialized plant marketplace, real-time chat notifications, comprehensive seller dashboards, integrated cart and favorites, secure payment methods, and enhanced user engagement features to improve the plant shopping experience.

Table 1-1 Literature Review

Platform	Weakness	Proposed Solution in Grow Mart
OLX	General marketplace; no plant-specific focus	Introducing a dedicated plant category, advanced filters, and buyer reviews
Daraz	No real buyer-seller chat; cluttered interface	Provide a clean UI with real-time notification functionality
Facebook Marketplace	Lacks order tracking and seller analytics	Add order lifecycle tracking and seller dashboards with analytics
Standard E-commerce	Too broad; no gardening-specific features	Dedicated plant categories with specialized search filters
Local Plant Shops	No online presence; limited reach	Digital marketplace accessible to all plant sellers

1.6 Analysis from Literature Review

The analysis highlights gaps in popular marketplace platforms like OLX, Daraz, and Facebook Marketplace. These include lack of plant-specific focus, limited buyer-seller communication, no structured order tracking, and insufficient seller analytics. Grow Mart solves these issues by offering:

1.6.1 Specialized Plant Commerce Platform

While most e-commerce sites offer general product listings, few support the specialized needs of plant buyers and sellers. Grow Mart enables users to register with specific roles (Buyer/Seller), browse categorized plant products, and enjoy a tailored shopping experience designed specifically for the gardening community.

1.6.2 Buyer-Seller Communication

Existing platforms often lack direct communication channels between buyers and sellers. Our solution uses Firebase Cloud Messaging to deliver real-time notifications about orders, payments, and delivery updates, ensuring both parties stay informed throughout the transaction process.

1.6.3 Order Tracking and Seller Analytics

Unlike generic platforms, Grow Mart includes a comprehensive order management system where sellers can update order statuses (Placed → Processing → Shipped → Delivered) and buyers can track their purchases in real-time. Sellers also have access to analytics dashboards showing sales performance and revenue trends.

1.6.4 Real-Time Notifications

A key feature that differentiates our platform is the integrated notification system. Both buyers and sellers receive instant push notifications via FCM for important events such as new orders, payment confirmations, delivery updates, and new reviews, keeping all users engaged and informed.

1.7 Methodology and Software Lifecycle

The development of Grow Mart followed an object-oriented, MVVM architecture-based, and incremental approach to ensure clarity, scalability, and efficiency.

1.7.1 Object-Oriented Methodology

Grow Mart is developed using Flutter, which naturally follows Object-Oriented design principles. Classes represent screens, models, controllers, and services, enabling easy maintainability and scalability. Reusability of widgets, components, and models was a key focus throughout develop.

1.7.2 MVVM Architecture

The MVVM (Model-View-ViewModel) architecture was adopted to cleanly separate the user interface from business logic and data models. The View layer displays all buyer and seller interfaces; the ViewModel layer handles application logic, validation, and communication with backend services; and the Model layer defines data structures like User, Product, Cart, and Order.

1.7.3 Incremental Approach

Using the Incremental Model, features were delivered in small, testable increments: Authentication → Products → Cart → Orders → Payments. Each module was tested before moving to the next, with feedback from the supervisor and testing results integrated continuously.

1.7.4 Rationale Behind the Selected Methodology

For the development of Grow Mart, a hybrid approach combining object-oriented methodology, MVVM architecture, and incremental development was selected to ensure efficient development, maintainability, and scalability.

This methodology ensures:

- **Modular Clarity:** Each component such as user authentication, product management, cart handling, payment processing, and notifications was developed as a separate, self-contained module. This structure allowed focused development, easier debugging, and future feature expansion.
- **Code Reusability**: By implementing object-oriented programming (OOP) and MVVM architecture, common functionalities such as user sessions, product attributes, and notification handling were abstracted into reusable classes and view models. This reduced code duplication and improved maintainability.
- **Continuous Testing**: Each module was tested independently after development. Incremental testing allowed for early identification of bugs, ensuring overall platform stability before integration.
- **Scalability**: The modular and MVVM design enables the system to adapt quickly. New features such as AI-based plant recognition or GPS delivery tracking can be integrated with minimal impact on the existing codebase.

1.7.5 Software Development Life Cycle (SDLC)

For Grow Mart, the Incremental Model was adopted within the SDLC framework. This model divides the development process into smaller, manageable parts (increments), each of which delivers a functional component of the final system.

This approach ensured timely delivery of core features and allowed flexibility in refining or expanding the platform based on testing outcomes and user feedback.

1.7.5.1 Planning and Requirements Gathering

During this initial phase, detailed requirements were collected based on the core idea: delivering a specialized plant e-commerce platform. The following key features were identified:

- User registration with role selection (Buyer/Seller)
- Product management for plant-related categories
- Shopping cart and order placement
- Payment processing via Stripe and COD
- Real-time push notifications via FCM
- Review and rating system
- Seller analytics dashboard

1.7.5.2 Design

A detailed object-oriented design was created for each module:

- **UI Design**: A clean and responsive user interface using Flutter's Material Design principles.
- **Backend Structure**: Modeled using MVVM concepts with Models, Views, and ViewModels.
- **Database Design**: Firebase Firestore collections for users, products, orders, carts, reviews, and notifications.
- **Security Design**: Firebase Authentication with role-based access control.

These designs ensured the system components were logically connected yet individually manageable.

1.7.5.3 Implementation

Each implementation followed an incremental rollout:

- Core modules like authentication and product management were developed first.
- Cart, checkout, payment, and notification modules were added in later stages.

Each module was developed as a separate component, with testing and integration following completion.

1.7.5.4 Testing

Multiple testing layers ensured the reliability of Grow Mart:

- **Unit Testing**: Verified individual functions like adding items to the cart, login validation, and product search.
- **Integration Testing**: Ensured smooth communication between UI, ViewModels, backend services, and payment gateways.
- **User Acceptance Testing (UAT)**: Real users tested the buying and selling experience, navigation, and responsiveness.

1.7.5.5 Deployment and Maintenance

Each increment was deployed to a testing environment for live testing. Once approved, it was prepared for production deployment.

- Deployment involved configuring Firebase services, setting up Firestore database, and preparing the APK for Android devices.
- Maintenance is ongoing, including bug fixes based on user feedback and feature enhancements.

Chapter 02

2.1 Problem Definition

The Grow Mart project aims to solve a growing issue in the plant and gardening e-commerce space: the lack of a specialized, dedicated platform for plant enthusiasts and small-scale sellers. While general e-commerce has revolutionized shopping convenience, it fails to provide the specialized features, categorization, and community focus needed for plant commerce, where product categories, care information, and seller specialization are critical. This project bridges that gap using modern mobile development technologies, role-based access control, and real-time data synchronization to create a dedicated plant marketplace that serves both buyers and sellers effectively.

2.2 Problem Statement

The main problem arises when plant lovers and small-scale gardening businesses seek to buy or sell plant-related products online but cannot find a centralized, specialized platform. Existing solutions are either too general, lacking plant-specific categories and filters, or fail to provide adequate buyer-seller communication, order tracking, and seller analytics. This often leads to frustration, missed opportunities, and inefficient transactions.

Grow Mart solves this problem by creating a dedicated mobile e-commerce platform specifically designed for plant-related products. The application provides role-based access for buyers and sellers, categorized product listings with smart search filters, secure payment processing through Stripe and COD, real-time order tracking, push notifications via Firebase Cloud Messaging, and seller analytics dashboards. This ensures more efficient transactions, enhances user confidence, and improves satisfaction for both plant buyers and sellers—ultimately creating a vibrant, specialized community for plant commerce.

2.3 Deliverables Requirements

2.3.1 Grow Mart Mobile Application

This is the main software product a mobile application built with Flutter and Firebase. The application offers:

- **User Registration & Authentication:** Users can sign up and log in securely using Firebase Authentication with role selection (Buyer/Seller).
- **Product Management:** Sellers can add, update, and delete product listings with images, prices, descriptions, and categories.
- **Product Browsing & Search:** Buyers can browse categorized products and use smart search filters.
- **Shopping Cart System:** Users can add products to their cart, adjust quantities, and manage items before checkout.
- **Payment Integration:** Secure checkout with Stripe for online payments and Cash on Delivery (COD) option.

- **Order Management:** Sellers can update order statuses; buyers can track their orders in real-time.
- **Review & Rating System:** Buyers can rate and review products after successful delivery.
- **Notification System:** Real-time push notifications via Firebase Cloud Messaging for order updates, payments, and reviews.
- **Seller Analytics:** Dashboard showing sales performance, revenue trends, and order statistics.

2.3.2 Documentation

User Documentation: This documentation is aimed at end-users of the Grow Mart platform. It provides clear, step-by-step instructions for navigating the interface, registering an account, browsing and searching products, managing the shopping cart, completing purchases, and tracking orders. Additionally, it explains how sellers can upload products, manage inventory, and view analytics.

Technical Documentation: This document is tailored for developers and technical stakeholders. It outlines the overall architecture of the system, including the Flutter frontend, Firebase backend services, MVVM design pattern, Firestore database schema, and Stripe payment integration. A breakdown of the source code structure, naming conventions, and dependencies is provided, along with configuration steps for setting up the development and deployment environment.

Admin Guide: The admin guide is created to support platform administrators in managing users, monitoring transactions, and performing essential system tasks. It includes instructions on accessing the admin panel, managing user roles, reviewing platform analytics, and maintaining system health.

2.3.3 Testing Reports

This section includes comprehensive documentation of all testing procedures and results gathered throughout the software development lifecycle. It consists of well-defined test cases for each functional module, detailing the inputs, expected outcomes, and actual outcomes. Reports from unit testing, integration testing, and user acceptance testing (UAT) are included.

2.3.4 Deployment Setup

The deployment section details the process of releasing the Grow Mart application for Android devices. It includes steps for configuring Firebase services, setting up Firestore database, configuring Firebase Authentication, enabling Firebase Cloud Messaging, and preparing the APK for distribution. This section also covers environment variables, security configurations, and any necessary optimizations for performance.

2.4 Development Requirements

2.4.1 Technologies and Tools

Frontend (Mobile Application):
The frontend is built using Flutter (Dart) with the MVVM architecture pattern. The user interface follows Material Design 3 principles for consistency and ease of use, providing a responsive layout that adapts to various screen sizes.

Backend Services:

The backend of Grow Mart utilizes Firebase services. Firebase Firestore serves as the real-time NoSQL database for storing user profiles, product listings, orders, carts, reviews, and notifications. Firebase Authentication manages secure user login and role-based access control. Firebase Storage handles product image uploads with URL references stored in Firestore documents. Firebase Cloud Messaging (FCM) delivers real-time push notifications to both buyers and sellers.

Payment Integration:
Stripe SDK is integrated for secure online payment processing. The system also supports Cash on Delivery (COD) as an alternative payment method. All payment communications occur over HTTPS, ensuring data confidentiality.

Development Tools:
Android Studio serves as the primary IDE for development. Git and GitHub are used for version control and source code management.

2.4.2 Hardware and Platform Requirements

- Android devices running version 8.0 (Oreo) and above
- Minimum RAM: 2 GB (4 GB recommended)
- Storage: 150 MB
- Processor: Quad-core or higher
- Connectivity: Wi-Fi or Mobile Internet (3G/4G)

2.4.3 Performance and Scalability

particularly during peak usage periods when many users may be browsing products or placing orders. Database query optimization strategies are used to efficiently manage product searches and filter operations. Real-time processing capabilities are integrated to support smooth notification delivery and order status updates, all while maintaining minimal latency.Firebase Firestore's real-time synchronization ensures optimal performance,

2.4.4 Security and Privacy

The platform implements strong user authentication through Firebase Authentication to prevent unauthorized access. Role-Based Access Control (RBAC) is used to define clear boundaries between buyers and sellers, allowing differentiated access to features. Sensitive user data is securely stored following Firebase's built-in security rules. Payment processing through Stripe is PCI-compliant, and Grow Mart never stores complete card details. All data transmission occurs over HTTPS to ensure confidentiality and prevent unauthorized access.

2.4.5 Development Milestones

The development milestones for Grow Mart are structured into five distinct phases. Phase 1 involves gathering requirements, defining user stories, and designing initial wireframes for the user interface. In Phase 2, core features such as user authentication, product management, and profile settings are developed. Phase 3 focuses on integrating shopping cart, checkout, payment processing, and notification systems. Phase 4 includes comprehensive testing, bug fixing, and deployment of a beta version for initial user feedback. Finally, Phase 5 covers the full deployment of the application, along with delivering user documentation and training resources.

2.4.6 Collaboration and Version Control

Git and GitHub are used for source code management and version tracking, ensuring organized development, seamless collaboration, and efficient tracking of changes throughout the lifecycle of the Grow Mart project.

2.5 Current System

At present, there is no single mobile e-commerce platform that fully integrates the features necessary for a comprehensive, specialized plant and gardening marketplace like the one developed in Grow Mart. Existing systems, such as general marketplaces (OLX, Daraz, Facebook Marketplace), provide basic buying and selling services, but they lack the integration of specialized features such as plant-specific categories, role-based dashboards, real-time order tracking, and seller analytics.

Typical marketplace platforms often offer static product listings or generic search, which fall short of delivering the specialized experience needed for plant buyers and sellers to make confident purchase decisions. Furthermore, many of them lack structured order management, buyer-seller communication channels, and analytics tools for sellers. Customer support features are usually limited to basic messaging, missing real-time notification capabilities that can improve satisfaction and reduce transaction friction.

In contrast, Grow Mart aims to combine the best elements of these systems while resolving their key limitations. It provides a dynamic, plant-specialized marketplace experience, an intuitive and real-time interface, robust seller tools, and personalized user features such as order tracking, ratings, and push notifications all designed to create a seamless, modern, and engaging plant.

Chapter 3

3.1 Requirement Analysis

The process of creating use case diagrams for Grow Mart involves a structured approach beginning with stakeholder analysis to understand the needs of buyers, sellers, and system administrators. This is followed by identifying and describing key use cases such as user registration, product browsing, cart management, payment processing, order management, and notification delivery. Relationships among these use cases are then established to show how different users interact with the system. Finally, the use case diagram is constructed to provide a clear visualization of system interactions.

This method ensures that all functional requirements are well defined and aligned with stakeholder expectations, offering a precise blueprint to guide developers in designing and implementing the platform effectively.

3.2 Use Case

The Grow Mart use case diagrams illustrate the interaction between three primary actors: the Buyer, the Seller, and the Administrator. This use case represents a comprehensive overview of the functionalities available on the platform for each user role.

Buyers can register or log in, browse and search plant-related products, add items to their shopping cart, proceed through checkout with Stripe or COD payment, track their orders, and submit reviews and ratings for purchased products.

Sellers are responsible for managing their product listings, handling incoming orders, updating order statuses, viewing sales analytics, and responding to customer reviews.

Administrators have elevated privileges that allow them to monitor users, manage products, review system logs, and oversee all transactions on the platform.

This use case serves as a blueprint for the system's core operations, ensuring both functional completeness and user engagement by integrating role-based access with comprehensive e commerce components.

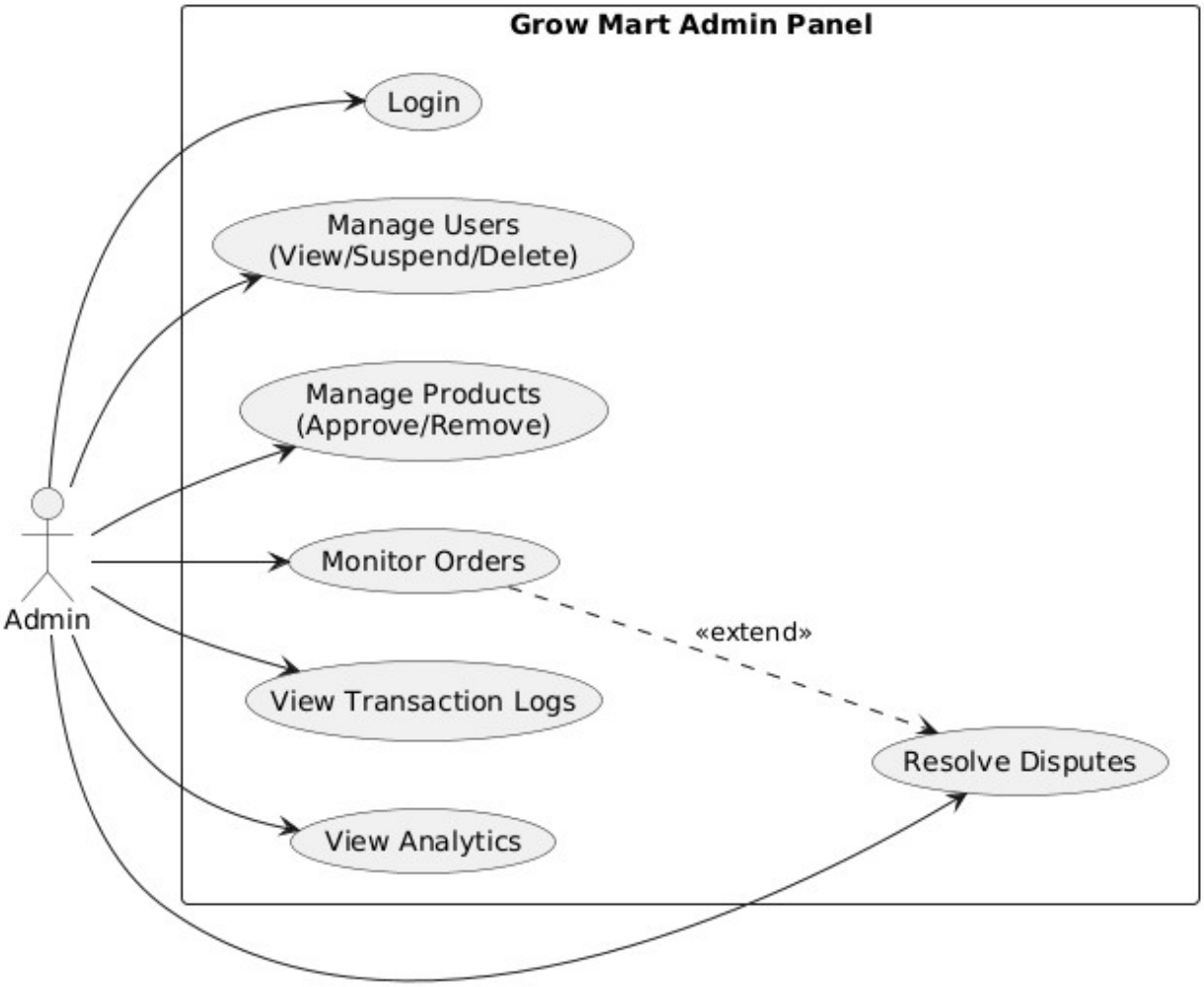

Figure 3.1-1 Use Case Diagram for Admin in Future

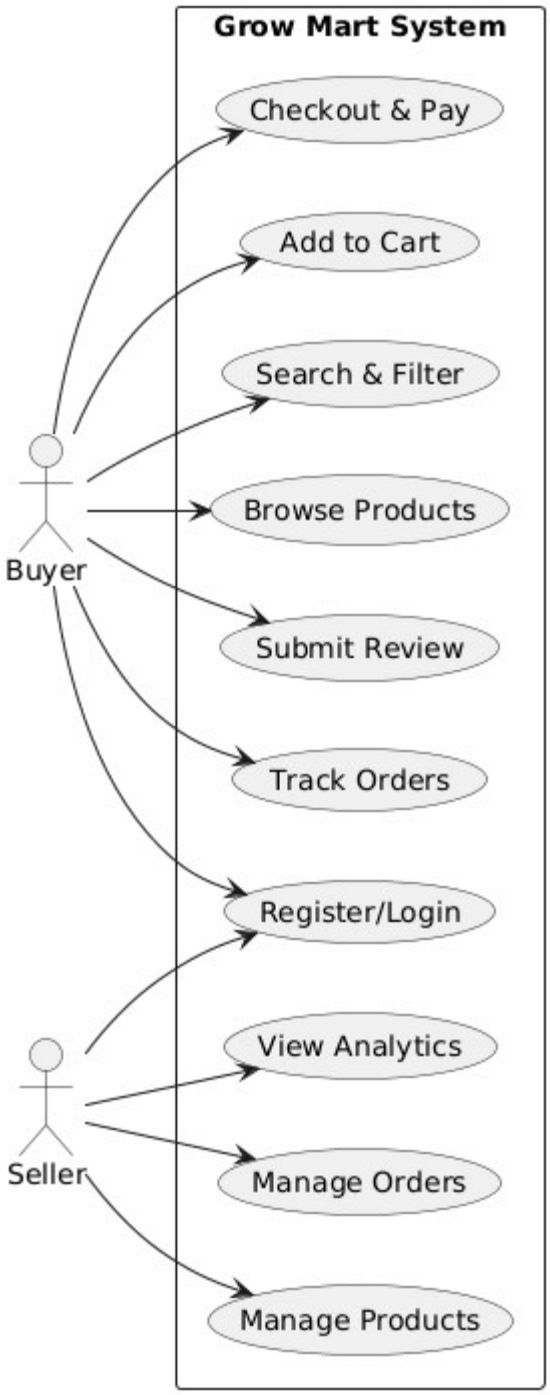

Figure 3.1- 2 Use Case Diagram For Buyer and Seller

3.3 Use Case Description

This use case diagram illustrates the interaction between the primary actors: the Buyer, Seller, and Administrator within the Grow Mart platform.
Buyers can register/log in to the platform, browse available plant products, search and filter listings, add products to their cart, proceed to checkout with Stripe or COD payment, track their orders, and submit reviews and ratings.
Sellers have elevated privileges that allow them to add, update, and delete product listings, manage incoming orders, update order statuses, view analytics, and monitor their sales performance.
Administrators can manage users, monitor transactions, review system logs, and ensure platform integrity.

3.3.1 Complete Platform Interaction For User Login

Table 3-1 User Login Use Case

Use Case ID	**UC-1**
Use Case Name	User Login
Actors	Buyer, Seller
Description	The user logs into the Grow Mart application using email and password via Firebase Authentication.
Trigger	User selects the "Login" option on the main screen.
Preconditions	User must already have a registered account.
Postconditions	User successfully accesses their respective Buyer or Seller dashboard.
Normal Flow	1. User opens app. 2. Selects Login. 3. Enters credentials. 4. System verifies login via Firebase Auth. 5. Redirects to role-based dashboard.
Alternative Flows	If credentials are incorrect, the system displays an error and prompts for re-entry.
Exceptions	Internet disconnected or Firebase service unavailable.

3.3.2 Complete Platform Interaction For Registration

Table 2-2 Registration Use Case

Field	Description
Use Case ID	UC-2
Use Case Name	Registration

Actors	Buyer, Seller
Description	Allow new users to register and create a Buyer or Seller account using Firebase Authentication.
Trigger	User selects "Create Account."
Preconditions	User is not already registered.
Postconditions	Account details are stored in Firebase Firestore, and user receives confirmation.
Normal Flow	1. User selects Register. 2. Enters details (name, email, password, role). 3. System validates data. 4. Account created in Firebase. 5. User redirected to role-based dashboard.
Alternative Flows	If email already exists, system shows "User Already Exists."
Exceptions	Missing required fields or weak internet connection.

3.3.3 Complete Platform Interaction For Browse and Search Products

Table 3-3 Browse and Search Products Use Case

Field	Description
Use Case ID	UC-3
Use Case Name	Browse and Search Products
Actors	Buyer
Description	Buyers can search for products using keywords, filters, and categories such as Indoor Plants, Outdoor Plants, Pots, Fertilizers, and Gardening Tools.
Trigger	Buyer enters search query or selects a category.
Preconditions	Buyer must be logged in.
Postconditions	Search results are displayed.
Normal Flow	1. Buyer navigates to "Search." 2. Enters keywords or applies filters. 3. System retrieves matching products from Firestore. 4. Results displayed in list view.
Alternative Flows	No products found—display "No Results Found" message.
Exceptions	Database connection error or timeout.

3.3.4 Complete Platform Interaction For Checkout and Payment

Table 3-4 Checkout and Payment

Field	Description
Use Case ID	UC-4
Use Case Name	Checkout and Payment
Actors	Buyer
Description	Buyers can make purchases using Stripe for online card payments or select Cash on Delivery (COD).
Trigger	Buyer selects "Proceed to Checkout."
Preconditions	Buyer has items in the cart.
Postconditions	Order confirmed and stored in Firestore. Buyer receives confirmation notification.
Normal Flow	1. Buyer reviews cart items. 2. Selects payment method (Stripe or COD). 3. System processes payment. 4. Order created in Firestore. 5. Confirmation notification sent.
Alternative Flows	Payment fails; system prompts retry or change payment option.
Exceptions	Payment gateway error or network failure.

3.3.5 Complete Platform Interaction For Add Product Listing

Table 3-5 Add Product Listing

Field	Description
Use Case ID	UC-5
Use Case Name	Add Product Listing
Actors	Seller
Description	Sellers can add new products by entering details like name, price, image, description, and category.
Trigger	Seller selects "Add Product."
Preconditions	Seller logged in and verified.
Postconditions	Product uploaded and visible in product listings.
Normal Flow	1. Seller enters product info (title, price, category, stock). 2. Seller uploads product images. 3. System validates data. 4. Images uploaded to Firebase Storage. 5. Product saved

	to Firestore. 6. Confirmation displayed.
Alternative Flows	Missing required fields or invalid image format.
Exceptions	Upload interrupted due to low internet connectivity.

3.3.6 Complete Platform Interaction For Order Management

Table 3-6 Order Management

Field	Description
Use Case ID	UC-6
Use Case Name	Manage Orders
Actors	Seller
Description	Sellers can view, update, or confirm orders placed by buyers.
Trigger	Seller selects "Orders" from dashboard.
Preconditions	Buyer has placed an order.
Postconditions	Order status updated (Placed → Processing → Shipped → Delivered). Buyer notified of change.
Normal Flow	1. Seller views list of incoming orders. 2. Seller selects order and updates status. 3. System updates status in Firestore. 4. FCM notification sent to buyer.
Alternative Flows	Seller cancels order due to product unavailability.
Exceptions	Order not found due to sync failure.

3.3.7 Complete Platform Interaction For Review and Rating

Table 3-7 Review and Rating

Field	Description
Use Case ID	UC-7
Use Case Name	Review and Rating
Actors	Buyer
Description	Buyers can rate purchased products (1-5 stars) and leave written feedback.
Trigger	Buyer selects "Leave Review" on a delivered order.
Preconditions	Buyer has completed the purchase and order is

	marked as Delivered.
Postconditions	Review stored in Firestore and displayed on product page.
Normal Flow	1. Buyer selects rating (stars). 2. Buyer enters comment. 3. System validates input. 4. Review saved to Firestore reviews collection. 5. Review displayed on product detail page.
Alternative Flows	Not applicable.
Exceptions	Attempt to rate without a completed order.

3.3.8 Complete Platform Interaction For Notifications

Table 3-8 Notifications

Field	Description
Use Case ID	UC-8
Use Case Name	Notifications
Actors	Buyer, Seller
Description	Both buyers and sellers receive real-time push notifications for order updates, payment confirmations, delivery changes, and new reviews through Firebase Cloud Messaging (FCM).
Trigger	System events such as new order placed, payment confirmed, delivery status updated, or review submitted.
Preconditions	User is logged in and has enabled notification permissions.
Postconditions	Notifications displayed on user's device and stored in-app notification log.
Normal Flow	1. System event triggered. 2. FCM generates push notification. 3. Notification appears on user's device. 4. User taps notification to view details.
Alternative Flows	If user has disabled notifications, alerts are saved in the in-app log only.
Exceptions	Firebase FCM delay or internet disconnection may cause temporary notification failure

3.4 Functional Requirements

These are the following modules that will be included in this system:

1. Buyer Module
2. Seller Module
3. Admin Module (Future Feature)

3.4.1 Buyer Module

- Buyers can register and log in using email and password via Firebase Authentication.
- Buyers can browse and search plant-related products using keywords, filters, and categories.
- Buyers can view detailed product information including images, descriptions, prices, stock status, and reviews.
- Buyers can add items to a shopping cart with quantity management.
- Buyers can update cart quantities or remove items before checkout.
- Buyers can proceed to checkout with Stripe (online card payment) or Cash on Delivery (COD).
- Buyers can view and track their order status (Placed → Processing → Shipped → Delivered).
- Buyers can view their complete order history.
- Buyers can rate and review products (1-5 stars) after successful delivery.
- Buyers receive real-time push notifications for order updates, payment confirmations, and delivery status changes.
- Buyers can submit feedback and contact support for any issues.

3.4.2 Seller Module

- Sellers can register and log in with role selection as Seller.
- Sellers can add, update, and delete product listings with images, descriptions, prices, categories, and stock quantities.
- Sellers can upload product images to Firebase Storage.
- Sellers can manage inventory and update product stock levels.
- Sellers can view and manage all incoming orders from buyers.
- Sellers can update order statuses (Placed → Processing → Shipped → Delivered).
- Sellers can view sales analytics including revenue trends, popular products, and order statistics.
- Sellers can view and respond to customer reviews.
- Sellers receive real-time push notifications for new orders and reviews.

3.4.3 Admin Module (Future Feature)

- Admins can monitor all users (Buyers and Sellers).
- Admins can manage product listings across the platform.
- Admins can view transaction logs and resolve disputes.
- Admins can review platform analytics and system health.

3.5 Non-Functional Requirements

Non-functional requirements are critical to ensuring that our Grow Mart platform performs well, is user-friendly, secure, reliable, and accessible across different devices. These requirements complement the functional aspects of the project by addressing key performance and quality attributes. The non-functional requirements for the Grow Mart mobile application are given below.

3.5.1 Usability

Grow Mart is designed to be simple, intuitive, and easy to use for all users, regardless of their technical background. The interface follows Flutter's Material Design principles to maintain a clean and modern look. Users can navigate smoothly through pages such as registration, product browsing, and checkout with minimal steps. Clear icons, readable text, and consistent layouts make the application accessible to everyone. The goal is to ensure that users can complete essential tasks like signing in, adding products, or placing orders efficiently and without confusion. Additionally, the design must be responsive to provide optimal functionality and appearance on various Android devices, including phones and tablets.

3.5.2 Performance

Performance is a core priority for Grow Mart. The system should respond quickly to user requests and maintain smooth functionality under normal and peak loads. Page transitions and data retrieval operations should take less than three seconds on a stable internet connection. The app utilizes Firebase Firestore's real-time synchronization to handle multiple users concurrently without delays. Optimized queries and caching techniques are used to ensure fast product searches and quick cart updates. The system must be scalable to handle a growing number of users and product listings without significant performance degradation.

3.5.3 Security

Grow Mart emphasizes data protection and user privacy. All user information and transactions are secured using Firebase Authentication and Stripe payment encryption. Data transmission is performed over HTTPS to ensure confidentiality. Role-based access control (RBAC) ensures that users (buyers, sellers) can only access authorized features. User input is validated to prevent NoSQL injection and other common attacks. The application follows Firebase's built-in security rules for secure data storage. Payment processing through Stripe is PCI-compliant, and Grow Mart never stores complete card details. All financial transactions must be secured using industry-standard encryption protocols.

3.5.4 Portability

Grow Mart is developed in Flutter, which makes it cross-platform and easy to deploy across multiple devices. Currently, the system supports Android 8.0 (Oreo) and above, with full compatibility for different screen sizes and resolutions. In future versions, Grow Mart can be ported to iOS and Web with minimal modifications. The APK size is lightweight and optimized for mid-range smartphones to ensure smooth installation and performance. The code base is modular and well-documented to facilitate easy updates and migrations.

3.5.5 Reliability

The system must always operate reliably, ensuring that users can complete their activities without interruption. Grow Mart uses Firebase's real-time database and auto-backup features to ensure data persistence and prevent data loss during system crashes or power failures. If an error occurs, the app should recover automatically and restore the previous session. The goal is to maintain 99% uptime with consistent availability for both buyers and sellers.

3.5.6 Maintainability

Grow Mart is built using the Model-View-ViewModel (MVVM) architecture, allowing easy updates, debugging, and module integration. All code is structured and well-documented for future developers. Firebase's cloud structure simplifies maintenance by centralizing data management and reducing manual server tasks. Error logs will be generated automatically for admin review, and the system should allow minor updates or bug fixes to be implemented within two working days.

Chapter 4

4.1 Design and Architecture

The design and architecture of Grow Mart are centered around developing a modular, scalable, and interactive mobile e-commerce platform specialized for plant and gardening products with role-based functionality. The frontend ensures a responsive and intuitive user experience for browsing products, managing the cart, completing purchases, and tracking orders. The backend, built using Firebase services, is responsible for secure data handling, real time synchronization, and well-structured module integration, including user authentication, product management, order processing, and notification delivery. This cohesive architecture supports smooth operations, intelligent product categorization, and personalized shopping experiences, enhancing both customer satisfaction and seller efficiency .

4.2 System Architecture

The system architecture of Grow Mart follows a layered, modular structure based on the MVVM pattern (Model-View-ViewModel). This structure cleanly separates the user interface, business logic, data models, and backend services.

The architecture follows a three-tiered structure: the View Layer, the ViewModel Layer, and the Model/Service Layer.

The View Layer (UI Screens) acts as the primary user interface, allowing buyers and sellers to interact with the platform. It displays all buyer and seller interfaces such as the home screen, product list, product details, cart, checkout, and seller dashboard. The View only shows data; it does not contain business logic.

The ViewModel Layer handles all application logic and state management. It processes user inputs, manages cart items, validates checkout data, communicates with Firebase services, and updates UI state. It acts as a bridge between the UI and the backend, enforcing all business rules and application workflows.

The Model/Service Layer handles communication with backend systems and external services:

- **Firebase Authentication** for secure user login and role-based access
- **Firebase Firestore** for real-time NoSQL data storage and synchronization
- **Firebase Storage** for product image hosting and management
- **Stripe Payment API** for secure online payment processing
- **Firebase Cloud Messaging (FCM)** for push notification delivery

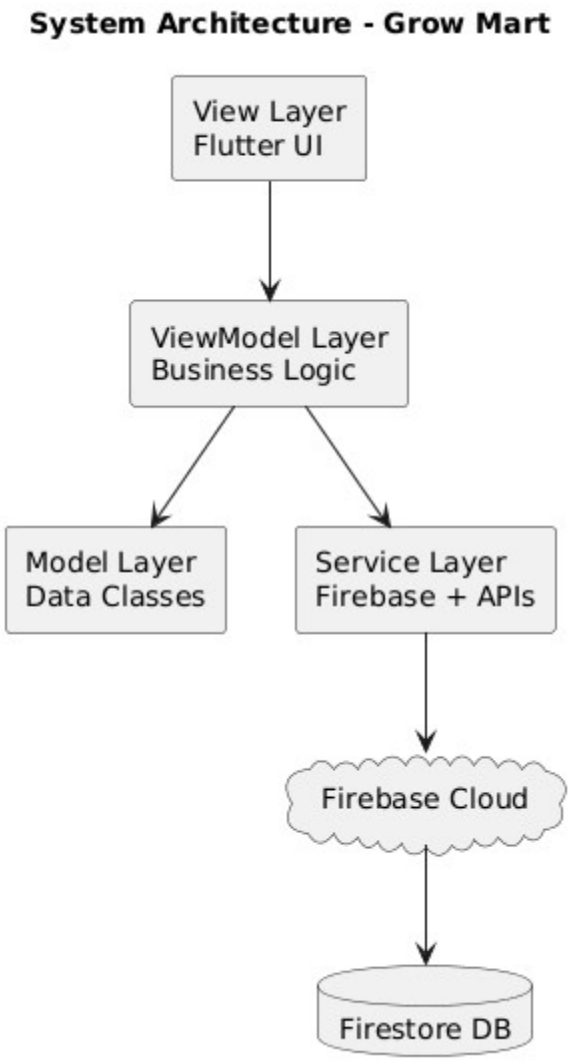

Figure 4.1-1 System Architecture

4.3 Data Representation

Given the interactive and feature-rich nature of the platform which includes role-based dashboards, product management, shopping cart functionality, real-time order tracking, customer reviews, and notification delivery the system relies on Firebase Firestore, a NoSQL cloud-hosted database, to manage data efficiently across various components.

Each core entity, such as users, products, orders, carts, reviews, and notifications, is stored in well-structured collections with clear document IDs and field definitions. This structure allows the system to accurately map user interactions with products and services, maintain consistent data flow, and ensure reliability during real-time operations.

The platform utilizes Firebase Firestore's querying capabilities to execute complex queries, data filters, and aggregations all of which are essential for dynamic search, personalized results, and seller analytics. This robust data layer is key to delivering a fast, secure, and personalized shopping experience.

While Grow Mart primarily relies on Firebase Firestore as its structured database, it also incorporates Firebase Storage for product images and JSON-structured data in API communications with Stripe. Firestore documents use key-value pairs that represent system data such as users, products, orders, carts, and reviews.

This hybrid data approach combines the reliability and efficiency of Firebase Firestore with the flexibility of cloud storage for images and external APIs for payment processing. By combining structured and semi-structured data models, the platform ensures robust performance, enhanced scalability, and the ability to adapt quickly to evolving features.

4.4 Process Flow/Representation

The Grow Mart platform process flow begins with user registration and authentication, providing secure access to features based on the selected role. Buyers can browse and search plant products, add items to their cart, proceed to checkout with Stripe or COD payment, and track their orders. Sellers can manage product listings, handle incoming orders, update delivery statuses, and view analytics. The system ensures smooth data flow between the frontend UI, ViewModels, backend Firebase services, and external payment APIs, supporting real-time synchronization, push notifications, and seamless management of all user interactions.

4.4.1 Process Flow for Buyer

The Buyer process flow illustrates the complete journey of a customer on Grow Mart. It begins with the user opening the app and landing on the welcome screen. The user registers or logs in with the Buyer role. The buyer browses product categories or uses search with filters. The buyer selects a product and views detailed information. The buyer adds the product to the shopping cart. The buyer reviews the cart and proceeds to checkout. The buyer selects a payment method (Stripe or COD). The system processes the payment and creates the order. The buyer receives an order confirmation notification.

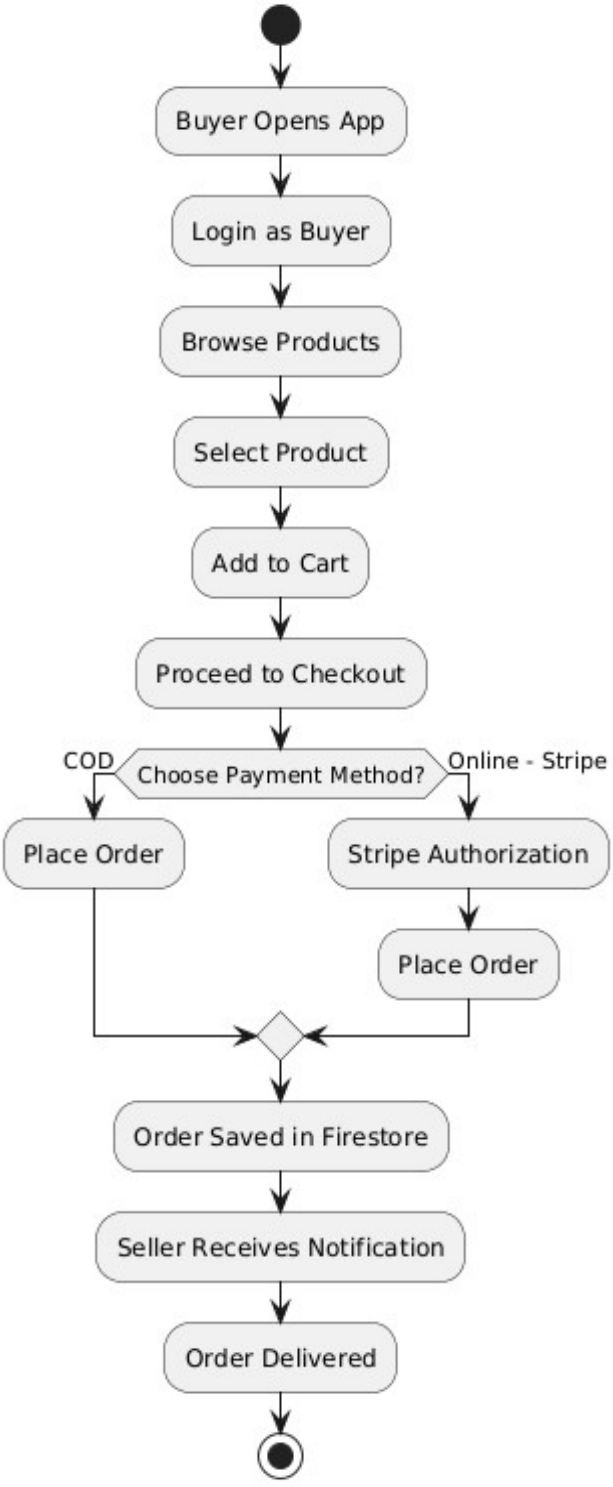

Figure 3.3-1 Process Flow – Buyer

4.4.2 Process Flow for Seller

The Seller process flow illustrates the complete journey of a seller on Grow Mart. The seller registers or logs in with the Seller role. The seller accesses the seller dashboard. The seller adds new products with images, descriptions, and prices. The seller manages existing product inventory and stock levels. The seller receives a notification for a new order. The seller views order details. The seller updates the order status (Placed → Processing → Shipped → Delivered). The seller views analytics showing sales and revenue trends through visual charts. The seller views and responds to product reviews. Each status update triggers a notification to the buyer, keeping them informed throughout the delivery process.

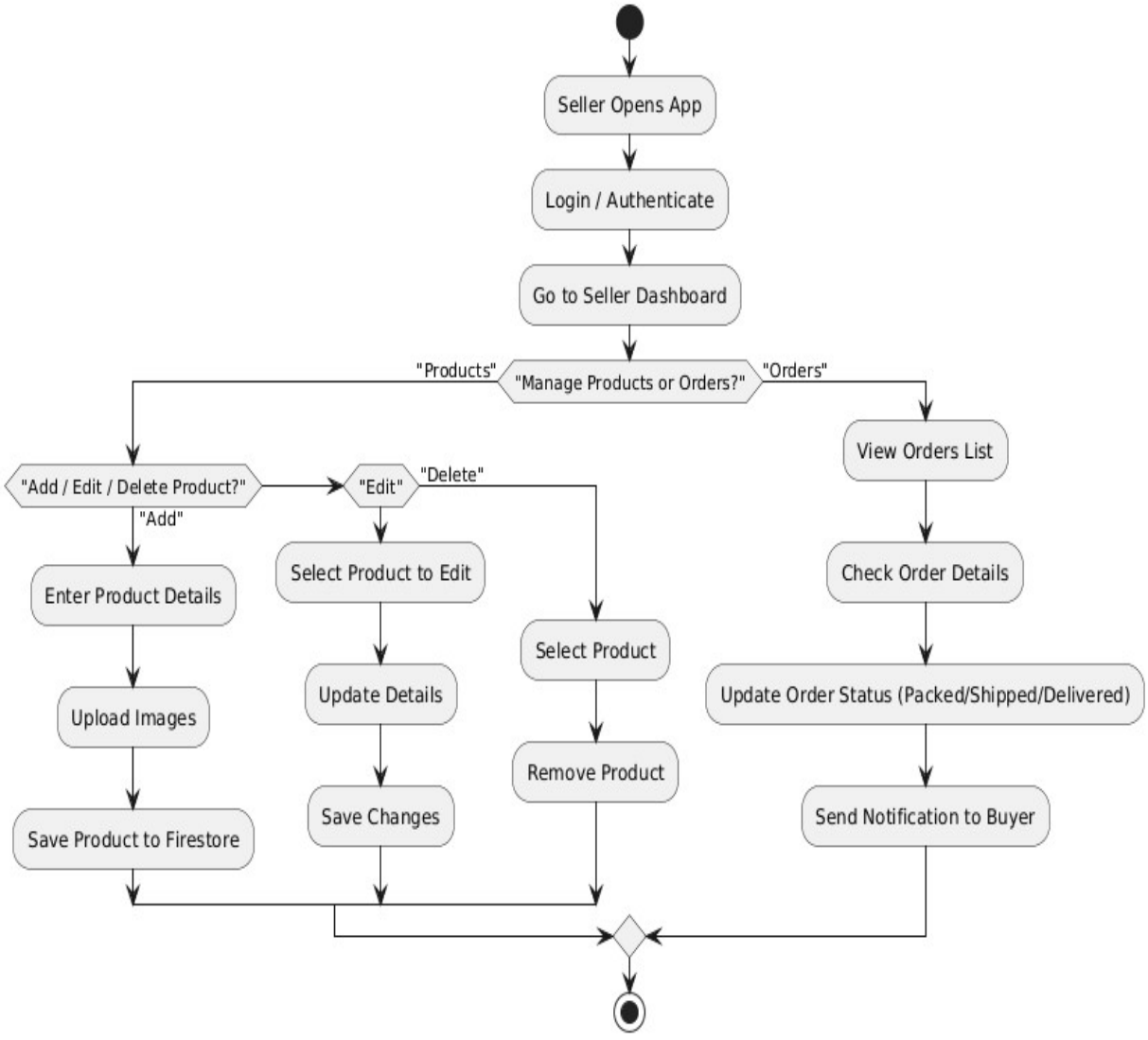

Figure 4.4-1 Process Flow – Seller

4.5 Design Models

The design models of Grow Mart provide a structured blueprint outlining how the system's core component such as user management, product catalog, order processing, payment handling, and notification delivery interact to deliver a seamless and intuitive plant shopping experience. It emphasizes a user-centric interface that allows easy navigation and role-based functionality. The model details the data flow between the Flutter frontend (View), MVVM ViewModels, and Firebase backend services, ensuring smooth integration of all components. Security measures for authentication and scalability considerations are embedded to support future growth, making the platform reliable, efficient, and engaging for users and administrators alike.

4.5.1 Sequence Diagram – Buyer Checkout and Stripe Payment

The Sequence Diagram for the Buyer Checkout process comprehensively illustrates the detailed interaction flow between the buyer and the system's key components during the payment process. It outlines each step the buyer takes, starting from initiating checkout from the Cart Screen.
The flow is as follows:
Buyer initiates checkout from the Cart Screen
Checkout ViewModel validates cart items and calculates total amount
ViewModel sends payment request to Payment Service
Payment Service communicates with Stripe API
Stripe processes payment and returns confirmation
On success, Order Service creates order document in Firestore
Order Service updates inventory quantities
Notification Service sends FCM confirmation to buyer
Notification Service sends FCM alert to seller about new order
Buyer is redirected to order confirmation screen
By mapping these interactions in sequence, the diagram provides a clear blueprint of the system's dynamic behavior, ensuring that all processes are logically ordered and optimized to meet user needs effectively and efficiently.

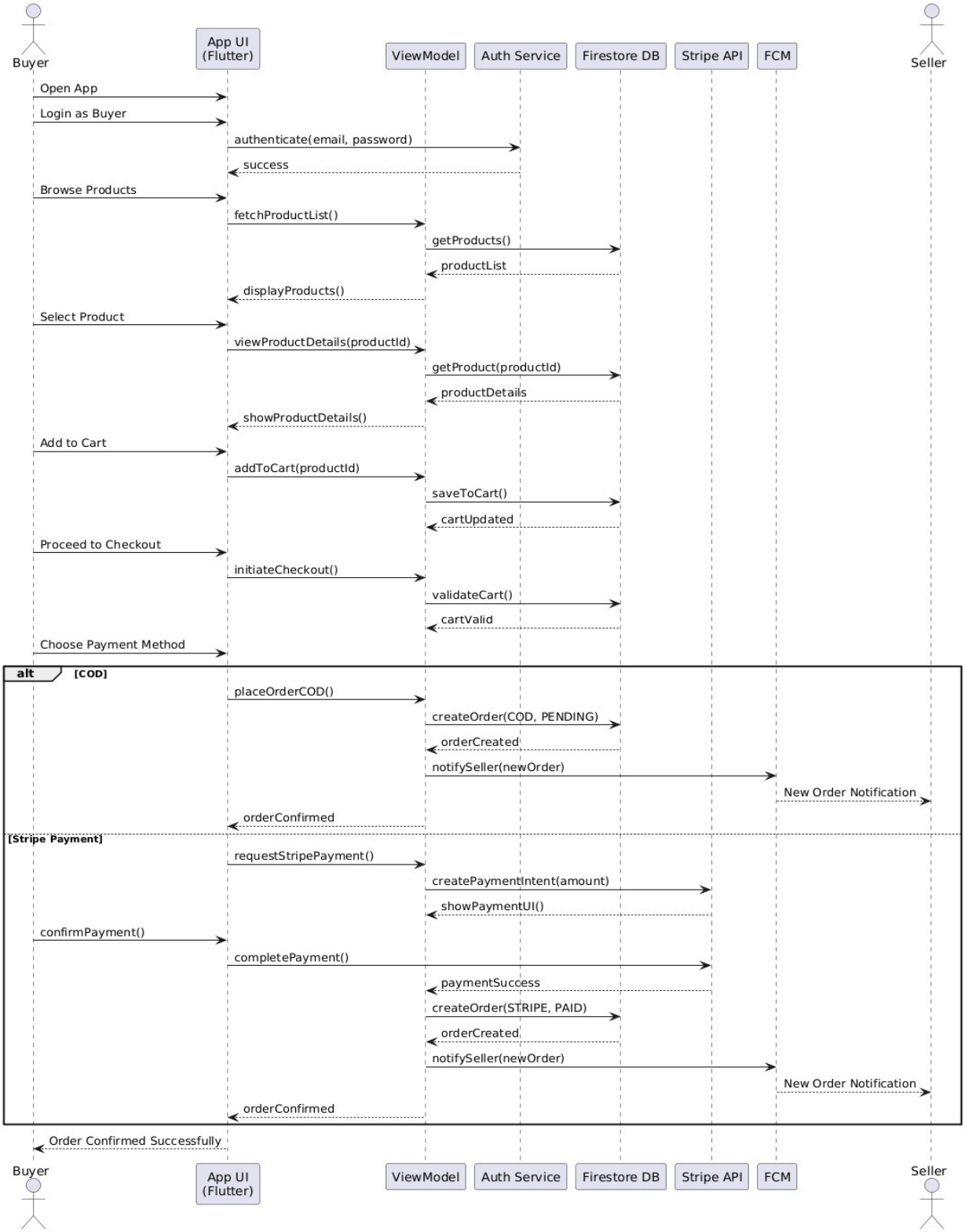

Figure 4.5-1 Sequence Diagram – Buyer

4.5.2 Sequence Diagram – Seller Order Management

The Sequence Diagram for the Seller Order Management process comprehensively illustrates the interaction flow between the seller and the system:
Seller receives push notification of new order via FCM
Seller opens Order Management screen from dashboard
Order ViewModel fetches order details from Firestore
Seller views complete order information (items, buyer details, total)
Seller updates order status (e.g., from Placed to Processing)
Order ViewModel validates the status transition
Updated status saved to Firestore orders collection
Notification Service sends status update FCM to buyer
Updated status reflected in seller's order dashboard
Process repeats for subsequent status updates (Processing → Shipped → Delivered)

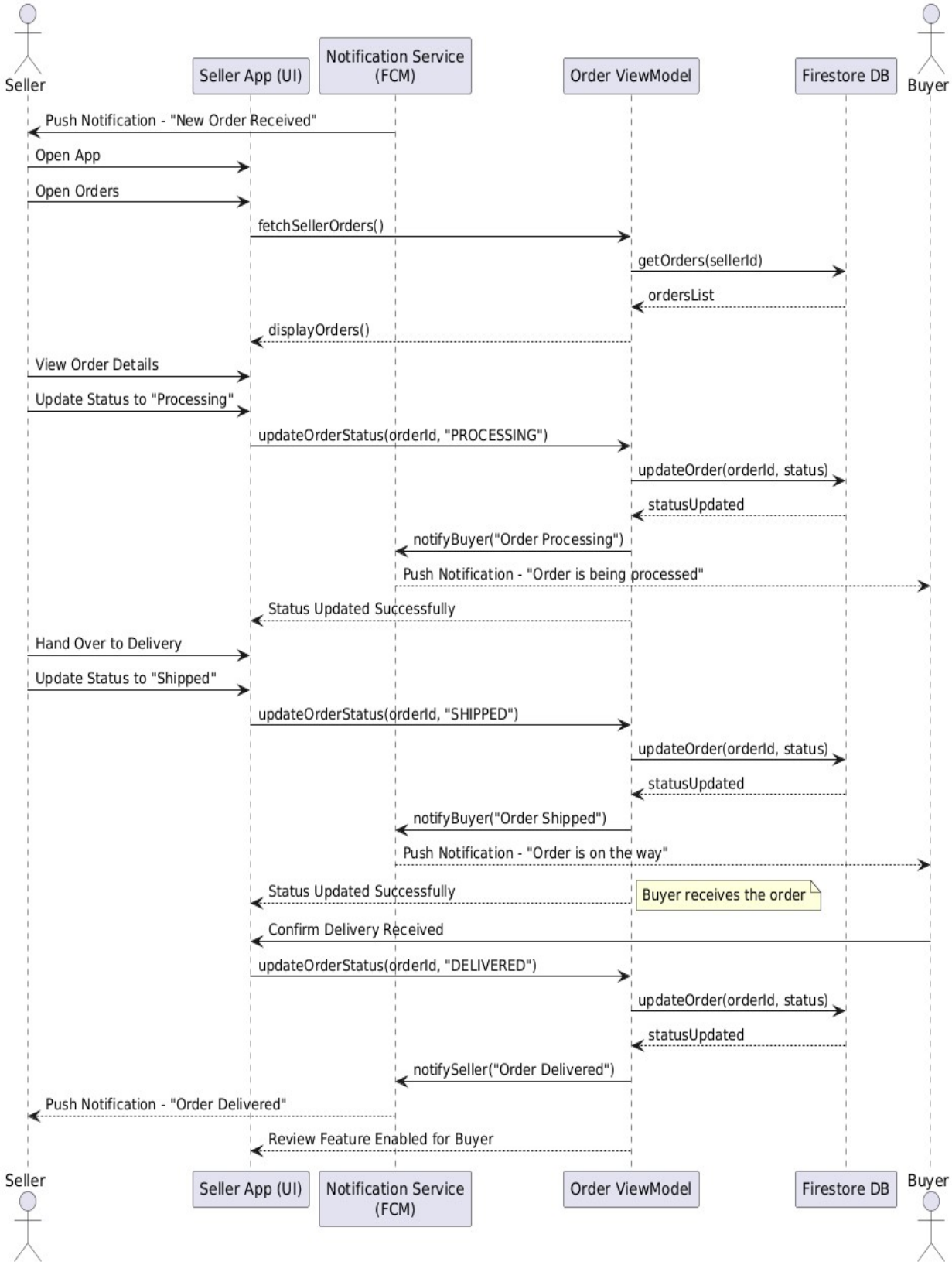

Figure 4.5-2 Sequence Diagram – Seller Order Managemen

4.5.3 Class Diagram

A Class Diagram is a type of UML (Unified Modeling Language) diagram used to represent the static structure of a system by showing its classes, attributes, methods (functions), and the relationships between objects. It is a foundational blueprint in object-oriented design and is especially useful for understanding how different parts of a system interact at a structural level.

The Class Diagram for the Grow Mart project provides a detailed visualization of the system's static architecture by illustrating its core classes, attributes, methods, and the relationships among them. It encapsulates the structure of key entities such as User, Product, Order, Cart, Review, and Notification.

Key classes include:

- **User**: Attributes include userId, name, email, password, phone, role (BUYER/SELLER), address. Methods include register(), login(), updateProfile().
- **Product**: Attributes include productId, sellerId, title, description, price, category, stock, images (List of URLs). Methods include addProduct(), updateProduct(), deleteProduct().
- **Cart**: Attributes include cartId, userId, items (List of CartItem containing productId and quantity), totalAmount. Methods include addItem(), removeItem(), updateQuantity(), clearCart().
- **Order**: Attributes include orderId, buyerId, sellerId, items (List of OrderItem), totalAmount, paymentMethod (COD/STRIPE), paymentStatus (PAID/PENDING/FAILED), orderStatus (PLACED/PROCESSING/SHIPPED/DELIVERED), createdAt. Methods include createOrder(), updateStatus(), cancelOrder().
- **Review**: Attributes include reviewId, productId, userId, rating (1-5), comment, createdAt. Methods include submitReview(), deleteReview().
- **Notification**: Attributes include notificationId, userId, title, message, type, isRead, createdAt. Methods include sendNotification(), markAsRead().

The diagram showcases associations (e.g., a User can place multiple Orders, a Seller can list multiple Products) and aggregations (e.g., an Order aggregates multiple OrderItems). This class diagram serves as a blueprint for the Firebase Firestore database schema and object-oriented programming logic, ensuring consistency across development and facilitating code maintenance.

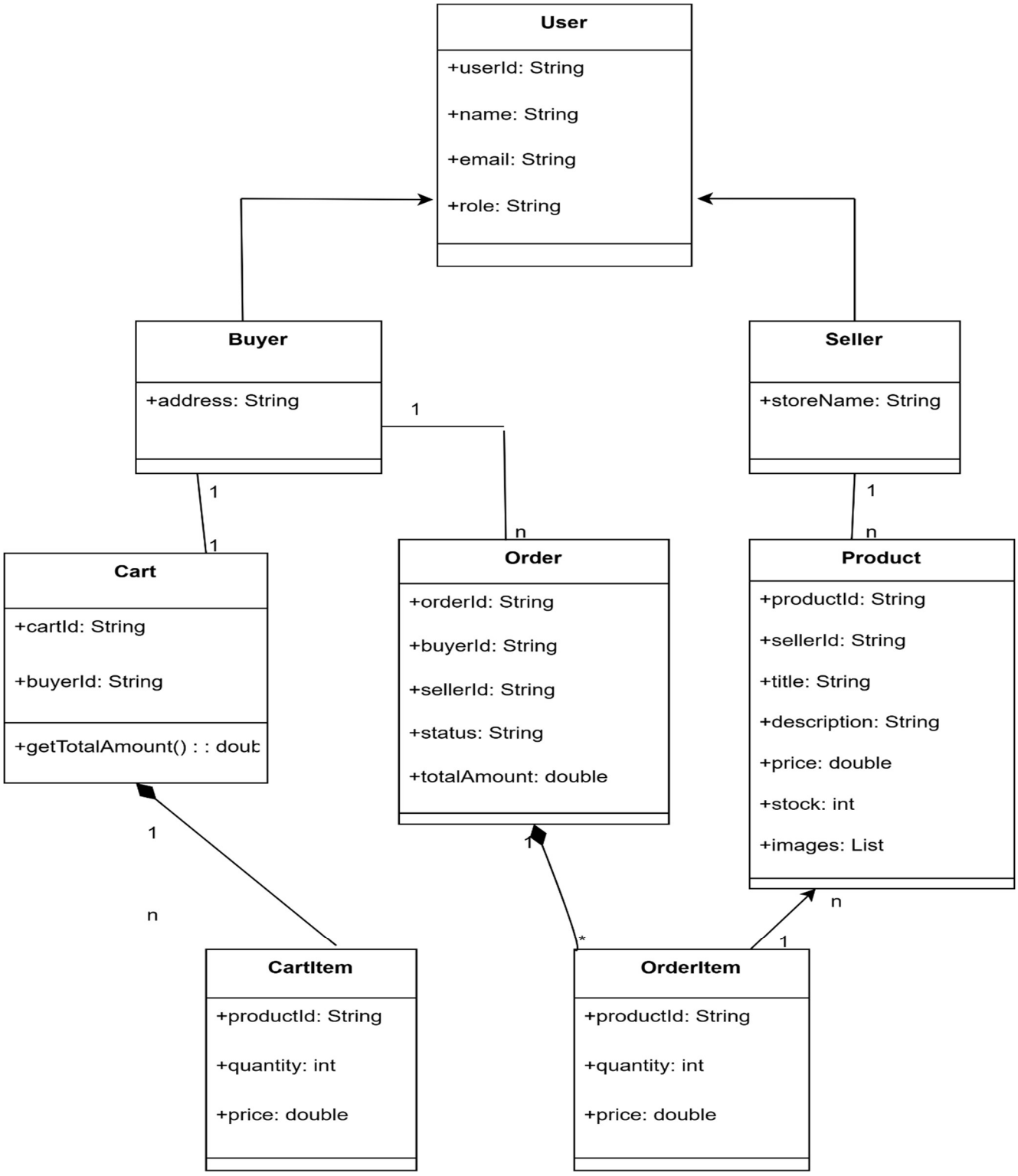

Figure 4.5-3 Class Diagram – Buyer and Seller

4.5.4 State Transition Diagram – Order Lifecycle

- The State Transition Diagram illustrates the complete lifecycle of an order in Grow Mart, showing how order states transition from creation to completion:

PLACED → PROCESSING → SHIPPED → DELIVERED

- **PLACED**: Order has been created by the buyer and is awaiting seller confirmation. Payment status may be PAID (Stripe) or PENDING (COD).
- **PROCESSING**: Seller has confirmed the order and is preparing the items for shipment.
- **SHIPPED**: Order has been dispatched for delivery to the buyer's address.
- **DELIVERED**: Order has been successfully received by the buyer. This state enables the review and rating functionality.

At any point before the SHIPPED state, an order can be CANCELLED by the seller due to product unavailability or other reasons. When cancelled, a notification is sent to the buyer and the inventory is restored.

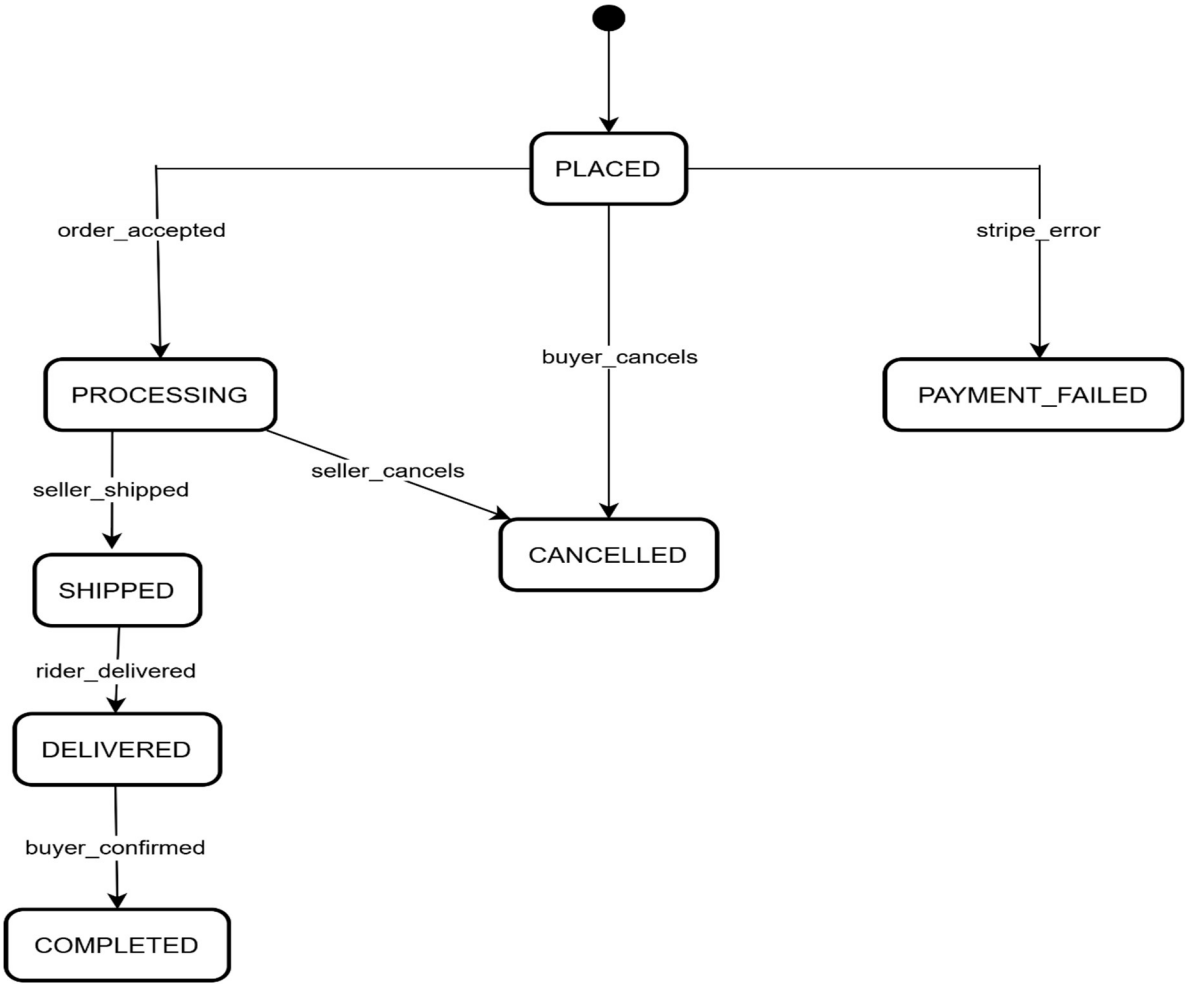

Figure 4.5-4 State Transition Diagram – Order Lifecycle

Diagrams (DFD)

Data Flow Diagrams provide a visual representation of how data moves through the Grow Mart system, showing the interactions between users, processes, and data stores.

DFD Level 0 Grow Mart Context Diagram: Shows the high-level overview of data flow between the Buyer, Seller, and the Grow Mart system as a single process. Buyers input registration data, orders, and reviews; they receive product listings, order confirmations, and notifications. Sellers input product listings and order updates; they receive order notifications and analytics data.

DFD Level 1 Checkout Process: Details the data flow during the checkout process including cart validation, shipping details collection, payment method selection, and order creation. The process flows from Cart Management to Checkout Processing, interacting with the Products data store for inventory verification and the Orders data store for order creation.

DFD Level 2 Payment Processing: Shows the detailed flow during payment processing including communication with Stripe API, payment verification, and order confirmation. The process splits between Stripe payment flow and COD flow, with payment results stored in the Orders data store and notifications dispatched via FCM.

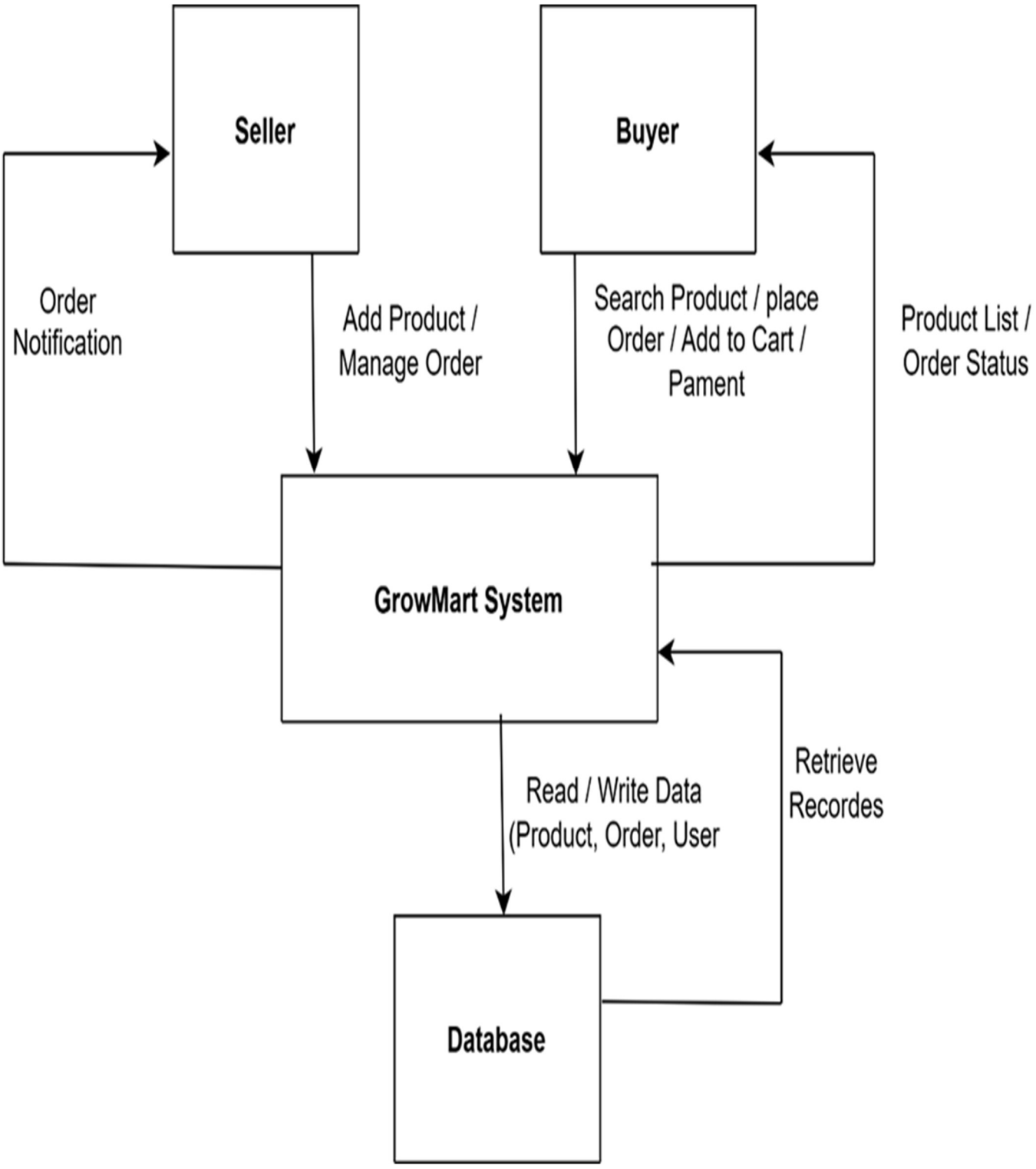

Figure 4.5-5 DFD Level 0 – Grow Mart

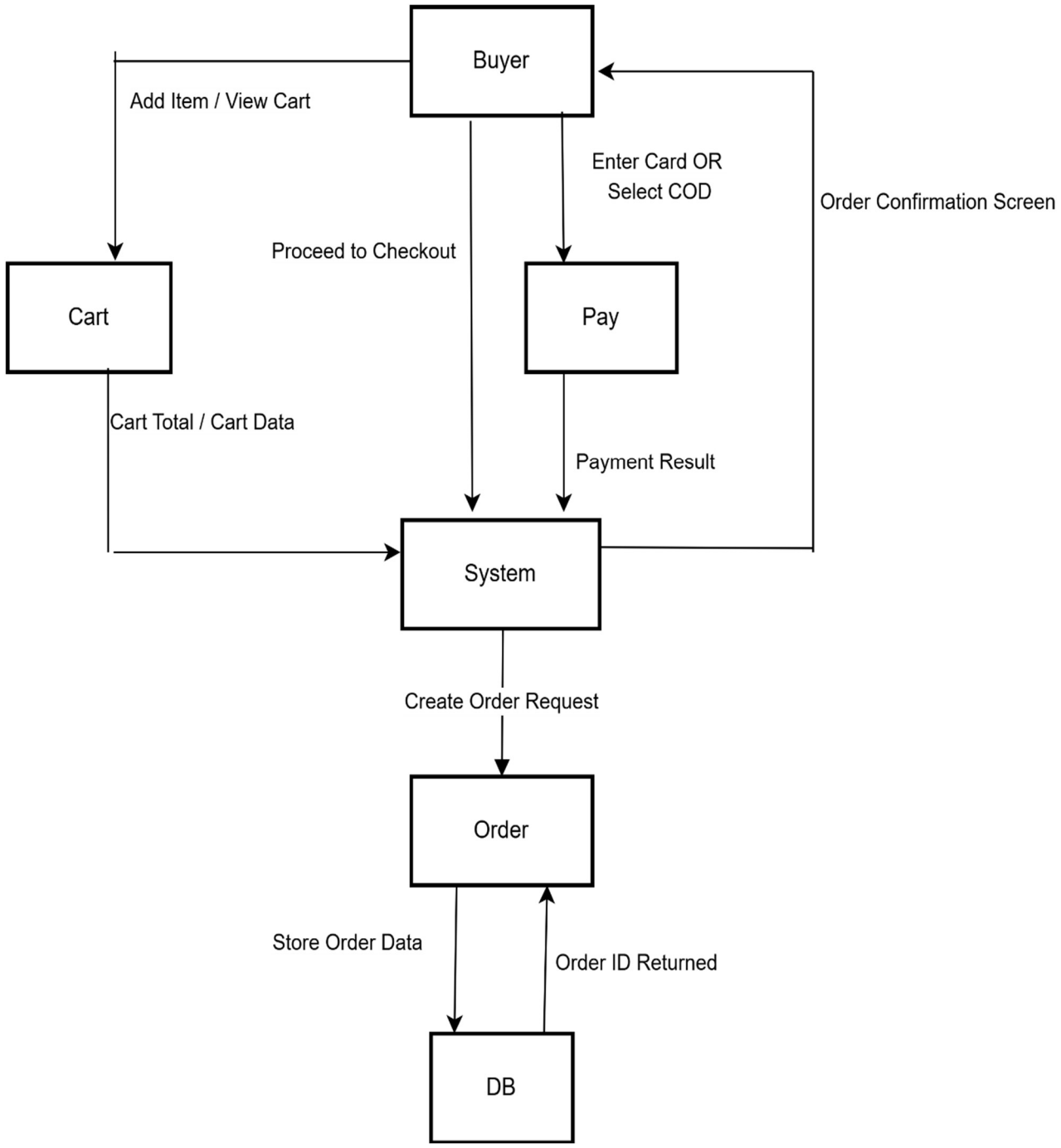

Figure 44.5-6 DFD Level 1 – Checkout Process

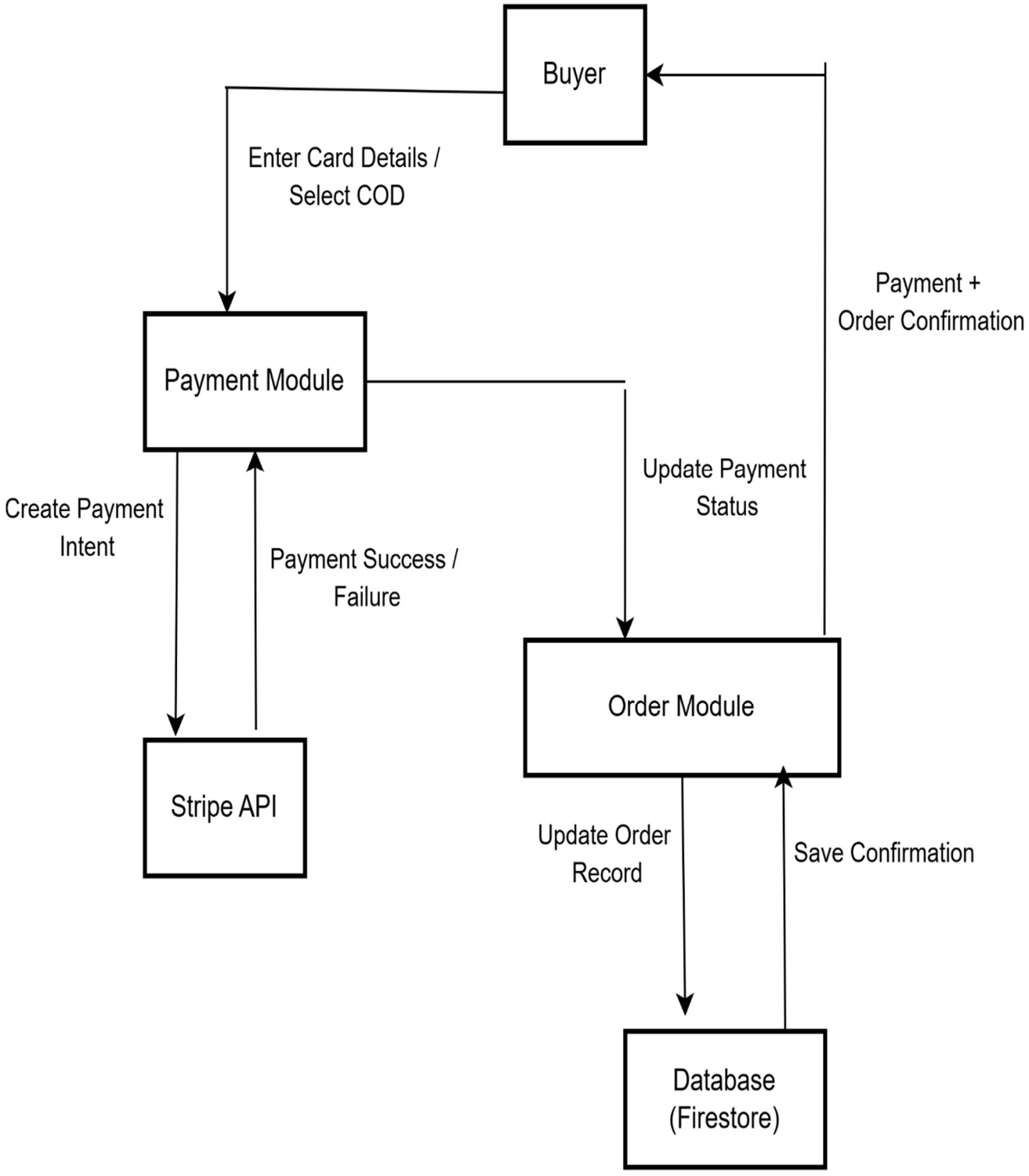

Figure 4.5-7 DFD Level 2 – Payment Processing

4.5.6 Component Architecture Diagram – MVVM + Firebase

This diagram illustrates how the MVVM architecture integrates with Firebase services and external APIs in the Grow Mart application:

- **UI Layer (View):** Flutter widgets and screens including Buyer Screens (Home, Product List, Product Detail, Cart, Checkout, Order Tracking) and Seller Screens (Dashboard, Product Management, Order Management, Analytics).
- **ViewModel Layer:** State management and business logic including AuthViewModel, ProductViewModel, CartViewModel, OrderViewModel, ReviewViewModel, and NotificationViewModel.
- **Repository Layer:** Data access abstraction that mediates between ViewModels and Firebase services.
- **Firebase Services:** Firebase Authentication (user management), Firestore Database (real-time data), Firebase Storage (image hosting), Firebase Cloud Messaging (push notifications).
- **External Services:** Stripe Payment API for secure transaction processing.

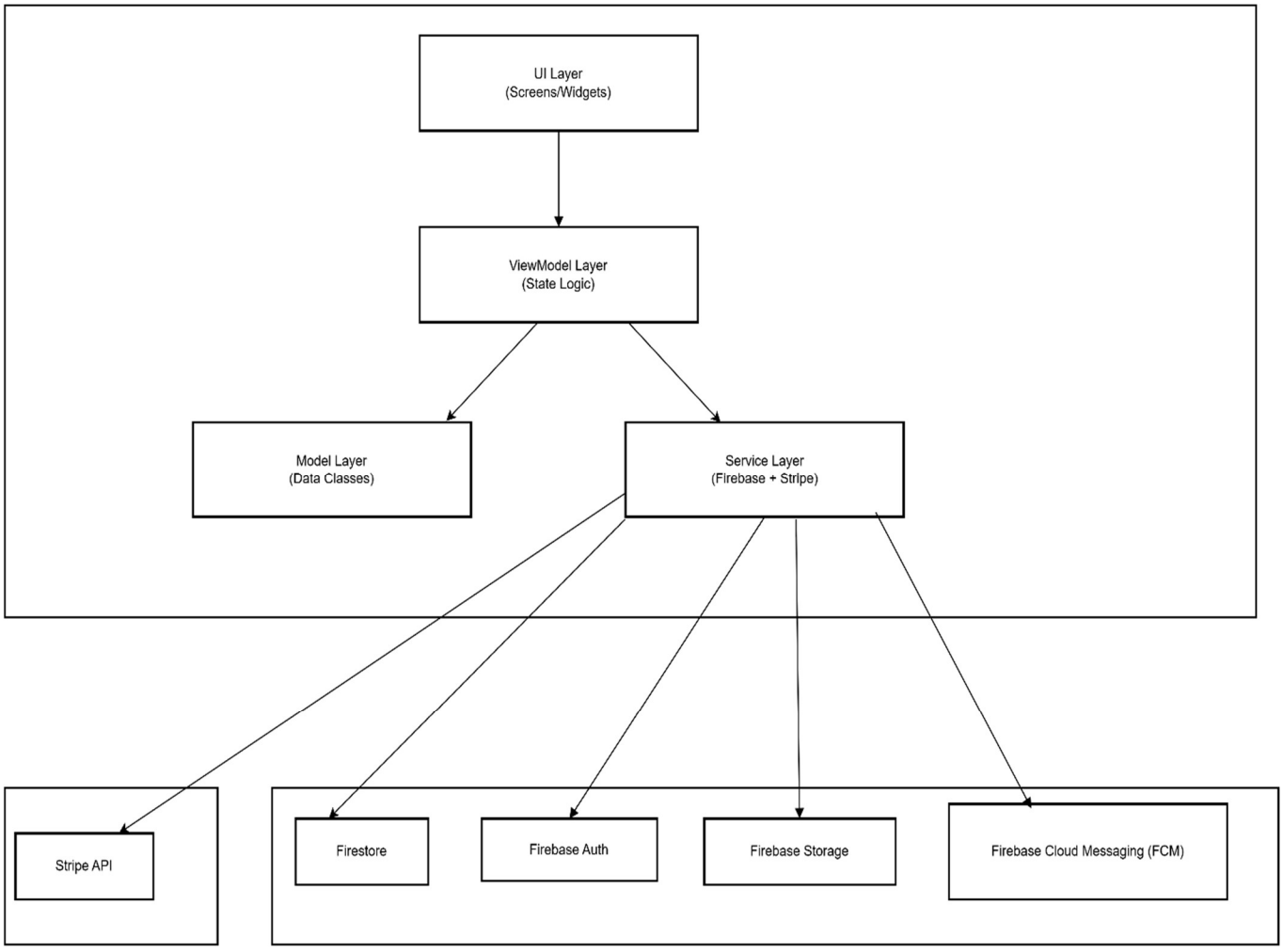

Figure 4.5-8 Component Architecture Diagram – MVVM + Firebase

4.6 Data Design

The data design of Grow Mart defines how the information domain of the system is transformed into structured data models. Since the application uses Firebase Firestore (NoSQL database), all major entities are stored as collections containing documents. Each document holds structured key-value pairs that represent system data such as users, products, orders, carts, reviews, and notifications.

As we discussed all the aspects of Grow Mart, now we will discuss the data design of our project. It describes the whole data structure of our application to ensure an efficient and secure system.

Grow Mart uses the following storage components:

Firebase Firestore (Main Database) Stores all dynamic application data. Collections include users, products, orders, carts, reviews, and notifications. Firestore provides real-time synchronization, ensuring all users see up-to-date information instantly.

Firebase Storage: Stores product images uploaded by sellers. Image URLs are referenced inside Firestore documents. This separates large binary data from the structured database for optimal performance.

Stripe (External Payment Storage) Stores payment intents and transaction records. Grow Mart never stores complete card details, maintaining PCI compliance.

4.6.1 Data Organization

User Information will store details about users, including their names, email addresses, passwords (securely encrypted via Firebase Auth), phone numbers, delivery addresses, and roles (BUYER or SELLER) to manage permissions effectively.

Seller Information will maintain details about sellers managing their product listings, including store information, contact details, and analytics preferences.

Product Information will store data about plant related products available in the system, such as product ID, seller ID, title, description, category, price, stock quantity, and image URLs.

Cart Information will include details about items added to a buyer's cart, such as product IDs, quantities, and the calculated total price.

Order Information will maintain records of buyer orders, including order ID, buyer ID, seller ID, product details with quantities, total price, payment method, payment status, order status, shipping address, and timestamps.

Review Information will store buyer ratings and comments linked to specific products and orders.

Notification Information will manage push notification records, tracking delivery status and read receipts for both buyers and sellers.

Database Security measures, such as Firebase Authentication and Firestore Security Rules, are implemented to protect sensitive information and enforce role-based access control.

4.6.2 Data Dictionary

The Data Dictionary for the Grow Mart project is a list of the names and attributes used within the system. These elements define the key components that will be implemented in the project. It serves as a reference for developers, designers, and stakeholders to ensure consistency in data usage and management. By standardizing data definitions, it helps streamline the development process and improve system efficiency. The dictionary includes details such as data types, formats, and relationships between different entities in the system.

- **User**: A person who uses the application to buy or sell plant-related products.

Attributes: userId, name, email, password (hashed), phone, role (BUYER/SELLER), address, createdAt.

- **Product**: Information about plant-related items available for sale.

Attributes: productId, sellerId, title, description, price, category (Indoor/Outdoor/Pots/Fertilizers/Tools), stock, images (List of URLs), createdAt, updatedAt.

- **Cart**: A temporary storage of items that a buyer intends to purchase.

Attributes: cartId, userId, items (List containing productId, quantity, price), totalAmount, updatedAt.

- **Order**: Details of a buyer's purchase and order history.

Attributes: orderId, buyerId, sellerId, items (List containing productId, title, quantity, price), totalAmount, paymentMethod (COD/STRIPE), paymentStatus (PAID/PENDING/FAILED), orderStatus (PLACED/PROCESSING/SHIPPED/DELIVERED), shippingAddress, createdAt, updatedAt.

- **OrderItem**: Individual items within an order.

Attributes: productId, title, quantity, price, subtotal.

- **CartItem**: Individual items within a cart.

Attributes: productId, title, quantity, price, subtotal.

- **Review**: Buyer feedback and ratings for purchased products.

Attributes: reviewId, productId, userId, orderId, rating (1-5), comment, createdAt.

- **Notification**: Push notification records for users.

Attributes: notificationId, userId, title, message, type (ORDER_UPDATE/PAYMENT/DELIVERY/REVIEW), isRead, createdAt.

- **Payment**: Information about the financial transaction associated with a buyer's purchase.

Attributes: paymentId, orderId, paymentMethod, amount, transactionId (from Stripe), paymentStatus, createdAt.

Chapter 5

5.1 Implementation

The implementation phase of the Grow Mart platform translates system design into a fully operational mobile e-commerce solution for plant and gardening products. This stage involves the practical construction of core functionalities including user authentication with role selection, product management, shopping cart, secure checkout via Stripe and COD, order processing with real-time tracking, review and rating systems, and push notification delivery. The development process is modular and iterative, ensuring that each component integrates smoothly into the broader platform.

Flutter is chosen as the frontend framework due to its cross-platform capabilities, rich widget library, and support for Material Design 3 principles. The MVVM architecture pattern is used to cleanly separate UI from business logic. Firebase services provide the backend infrastructure, including Firestore for real-time NoSQL data storage, Authentication for user management, Storage for product image hosting, and Cloud Messaging for instant push notifications.

Key modules include user registration and authentication with role-based dashboards, product browsing with category filtering and smart search, cart management with quantity controls, checkout via Stripe or Cash on Delivery, order tracking with lifecycle status updates, review and rating system, seller analytics dashboard, and notification delivery via Firebase Cloud Messaging.

5.2 Algorithms

5.2.1 User Authentication

Function: Manages registration and login for buyers and sellers using Firebase Authentication.
Input: Email, password, role selection
Output: Authenticated session with role-based dashboard access
Pseudocode:
BEGIN

- IF login form submitted THEN
- VALIDATE email and password fields
- CALL Firebase Authentication signInWithEmailAndPassword()
- IF credentials valid THEN
- FETCH user role from Firestore users collection
- REDIRECT to role-based dashboard (Buyer or Seller)
- ELSE
- DISPLAY error message "Invalid credentials"
- ENDIF
- IF registration form submitted THEN
- VALIDATE name, email, password, and role
- CALL Firebase Authentication createUserWithEmailAndPassword()
- IF registration successful THEN

- SAVE user profile to Firestore users collection
- SEND email verification
- REDIRECT to role-based dashboard
- ELSE
- DISPLAY error message
- ENDIF
- END

5.2.2 Product Inventory Management

Function: Enables sellers to add, update, and delete product listings with image uploads.
Input: Product details (title, description, price, category, stock, images)
Output: Updated product catalog visible to buyers
Pseudocode:

- BEGIN
- IF seller submits add product form THEN
- VALIDATE all input fields (title, price, category, stock)
- IF images selected THEN
- UPLOAD images to Firebase Storage
- GET download URLs for all images
- ENDIF
- CREATE product document in Firestore products collection
- SET sellerId, title, description, price, category, stock, images, createdAt
- DISPLAY success confirmation
- IF seller updates product THEN
- UPDATE corresponding document in Firestore products collection
- IF seller deletes product THEN
- DELETE images from Firebase Storage
- REMOVE document from Firestore products collection
- END

5.2.3 Cart and Checkout System

Function: Manages product selection, cart updates, and order placement for buyers.
Input: Product ID, quantity, payment method
Output: Order confirmation and notification
Pseudocode:

- BEGIN
- IF buyer adds product to cart THEN
- CHECK if product exists in Firestore products collection
- VERIFY requested quantity is available in stock
- ADD item to cart with productId, title, quantity, price

- CALCULATE updated cart total
- SAVE cart to Firestore carts collection
- IF buyer proceeds to checkout THEN
- VALIDATE cart has items and quantities are valid
- DISPLAY order summary with total amount
- IF payment method is Stripe THEN
- CREATE Stripe payment intent via Stripe API
- PROCESS payment
- IF payment successful THEN
- SET paymentStatus to PAID
- CREATE order document in Firestore orders collection
- UPDATE product stock quantities
- CLEAR buyer's cart
- SEND FCM notification to seller
- SEND confirmation notification to buyer
- ELSE
- DISPLAY payment failure message
- ENDIF
- IF payment method is COD THEN
- SET paymentStatus to PENDING
- CREATE order document in Firestore orders collection
- UPDATE product stock quantities
- CLEAR buyer's cart
- SEND FCM notification to seller
- SEND confirmation notification to buyer
- ENDIF
- END

5.2.4 Order Management

Function: Allows sellers to view and update order statuses through the order lifecycle.
Input: Order ID, new status
Output: Updated order status with buyer notification
Pseudocode:

- BEGIN
- IF seller opens order management THEN
- FETCH all orders for seller from Firestore orders collection
- DISPLAY orders with current status and details
- IF seller updates order status THEN
- VALIDATE status transition is allowed
- (PLACED → PROCESSING → SHIPPED → DELIVERED)

- UPDATE orderStatus field in Firestore order document
- UPDATE updatedAt timestamp
- SEND FCM notification to buyer with new status
- IF status changed to SHIPPED THEN
- INCLUDE delivery details in notification
- ENDIF
- IF status changed to DELIVERED THEN
- ENABLE review and rating option for buyer
- ENDIF
- IF seller cancels order THEN
- UPDATE orderStatus to CANCELLED
- RESTORE product stock quantities
- SEND cancellation notification to buyer
- END

5.2.5 Review Management

Function: Allows buyers to submit ratings and reviews for delivered products.
Input: Rating (1-5 stars), review comment text
Output: Stored review displayed on product page
Pseudocode:

- BEGIN
- IF buyer submits review THEN
- VERIFY buyer has completed this order
- CHECK order status is DELIVERED
- VALIDATE rating is between 1 and 5
- VALIDATE comment is not empty
- CREATE review document in Firestore reviews collection
- SET productId, userId, orderId, rating, comment, createdAt
- UPDATE product average rating in products collection
- DISPLAY review on product detail page
- SEND FCM notification to seller about new review
- END

5.2.6 Admin Product and User Management (Future Feature)

Function: Enables admin to monitor and manage platform users and products.
Input: Admin panel actions (view, approve, delete)
Output: Updated database records
Pseudocode:

- BEGIN
- IF admin logs in THEN
- GRANT access to admin dashboard
- FETCH all users from Firestore users collection
- FETCH all products from Firestore products collection
- FETCH all orders from Firestore orders collection
- ALLOW user management (view, suspend, delete)
- ALLOW product management (view, approve, remove)
- ALLOW order monitoring and dispute resolution
- UPDATE database accordingly
- END

5.3 External Services and APIs

The Grow Mart platform leverages Firebase services and third-party APIs to deliver an intuitive and interactive plant e-commerce experience without relying on unnecessary external dependencies.

The platform is built using Flutter for the frontend and Firebase for backend services. Firebase Authentication manages user registration and login with role-based access control. Firestore Database provides real-time NoSQL data storage and synchronization across all users. Firebase Storage handles product image hosting with URL references stored in Firestore documents. Firebase Cloud Messaging (FCM) delivers instant push notifications for orders, payments, and reviews.

Stripe SDK is integrated for secure online payment processing, supporting multiple card types and providing PCI-compliant transaction handling. The system also supports Cash on Delivery (COD) as an alternative payment method.

Although Grow Mart does not support real-time chat or social logins in the current version, it focuses on delivering a smooth, responsive, and privacy-respecting user experience, prioritizing in-house functionalities to ensure platform independence and data security.

Table 5-1 External Services and APIs

Component	Technology / Tool	Purpose / Description
Frontend Framework	Flutter (Dart)	Cross-platform mobile UI development with Material Design 3 principles
Backend Services	Firebase (Google)	Handles serverless backend

		logic, authentication, database, storage, and notifications
Authentication	Firebase Auth	Manages user registration, login, email verification, and session handling without third-party OAuth
Database	Firebase Firestore	Real-time NoSQL database for storing user data, product catalog, orders, carts, reviews, and notifications
Image Storage	Firebase Storage	Hosts product images uploaded by sellers with secure access controls
Push Notifications	Firebase Cloud Messaging (FCM)	Real-time push alerts for order updates, payment confirmations, delivery changes, and new reviews
Payment Processing	Stripe SDK	Secure online payment integration supporting multiple card types with PCI compliance
Data Visualization	MPAndroidChart	Chart library for displaying sales analytics, revenue trends, and order statistics on seller dashboard
PDF Generation	Intext PDF Library	Invoice generation for order receipts that sellers and buyers can download
Development IDE	Android Studio	Primary integrated development environment for Flutter app development and debugging
Security	Firebase Security Rules + CSRF Protection	Ensures secure user sessions, role-based data access, and protection against common vulnerabilities

5.4 User Interface

The Grow Mart user interface (UI) is designed with simplicity and user-friendliness in mind, featuring a clean, intuitive layout that makes it easy for both buyers and sellers to navigate the platform. The interface follows Flutter's Material Design 3 principles to maintain a modern, consistent look across all screens.

5.4.1 Buyer Home Screen

This screen is the main gateway for buyers using Grow Mart. The layout is designed to be welcoming and user-friendly, instantly showcasing product categories such as Indoor Plants, Outdoor Plants, Pots, Fertilizers, and Gardening Tools. Prominent options for login and signup encourage user engagement, while intuitive navigation directs buyers to browse products, access their cart, search with filters, and track their orders. Featured products and new arrivals are displayed to attract buyer interest.

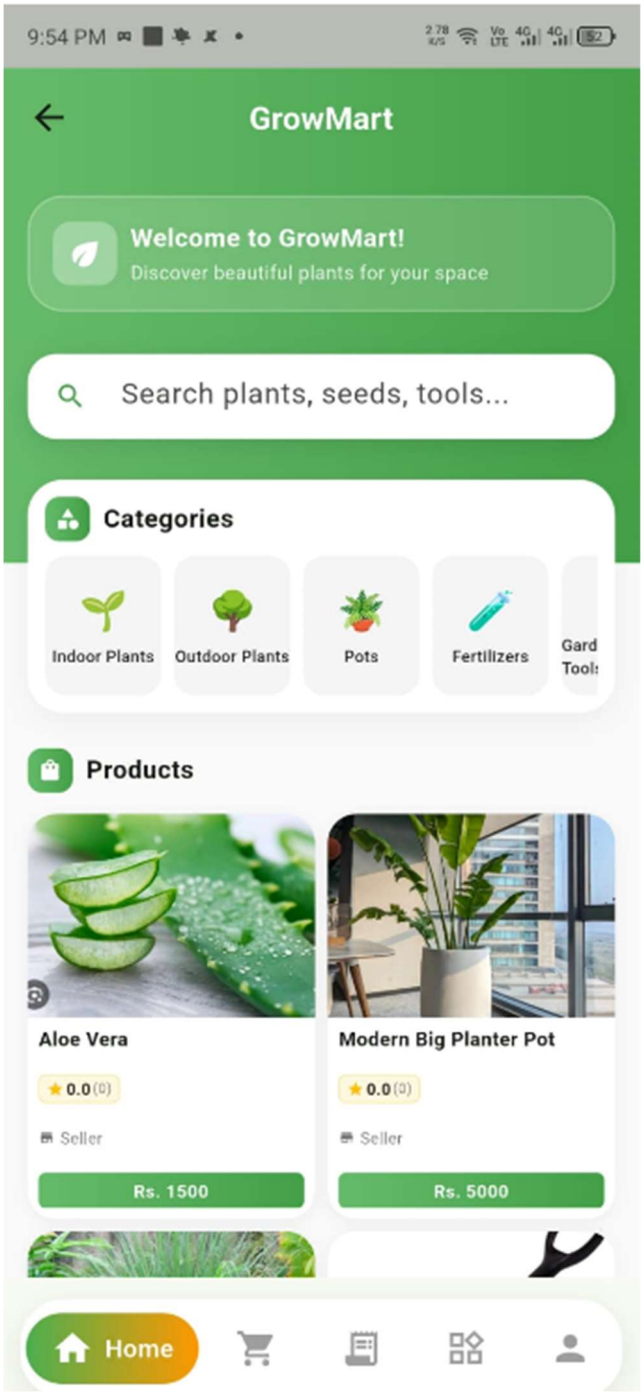

Figure 5.4-1 Buyer Home Screen

5.4.2 Product Catalog and Search

Displays a browsable and searchable list of plant-related products available on the platform. Buyers can scroll through categorized product listings
, apply filters for plant type, price range, and category, and use keyword search to find specific items. Each product card shows the image, title, price, and
average rating. Smart filters help narrow down the selection based on buyer preferences.

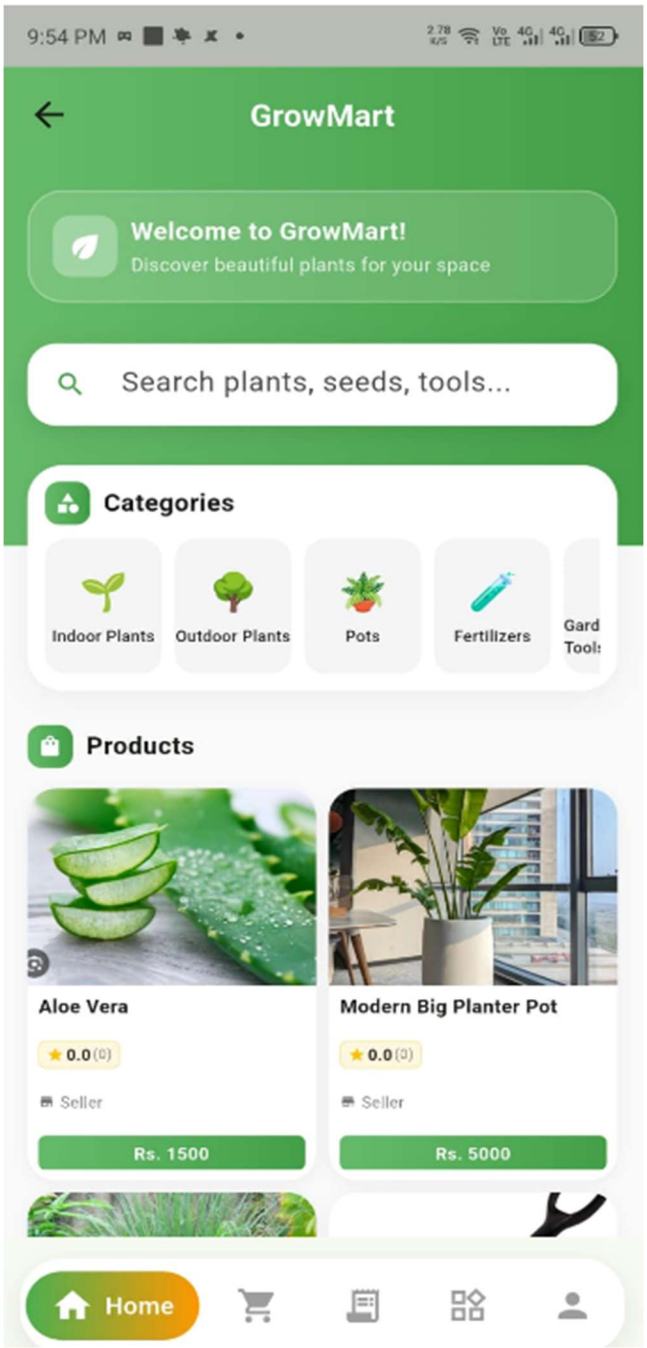

Figure 5.4-2 Product Catalog Screen

5.4.3 Product Detail Page

Displays detailed information about a selected plant product including multiple images, complete description, price, stock availability, category tags, seller information, and customer reviews. Buyers can view product specifications, read existing reviews, see the average rating, and add the item to their cart or wishlist directly from this page. The "Add to Cart" button is prominently placed for easy access.

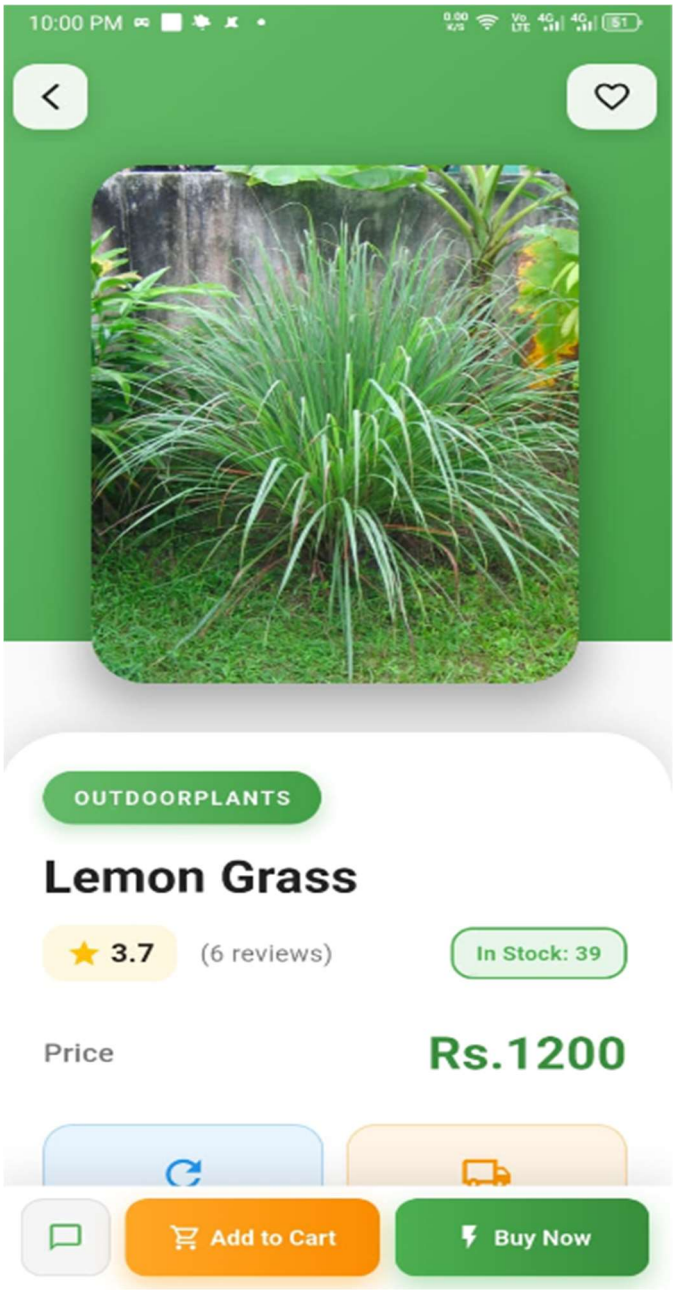

Figure 5.4-3 Product Detail Page

5.4.4 Shopping Cart

Allows buyers to view, manage, and proceed with selected items before checkout. The cart screen displays all added products with their images, titles, individual prices, selected quantities, and subtotals. Buyers can increase or decrease quantities, remove items, and see the calculated total amount. A "Proceed to Checkout" button guides buyers toward order completion. Empty cart states display helpful messages encouraging continued shopping.

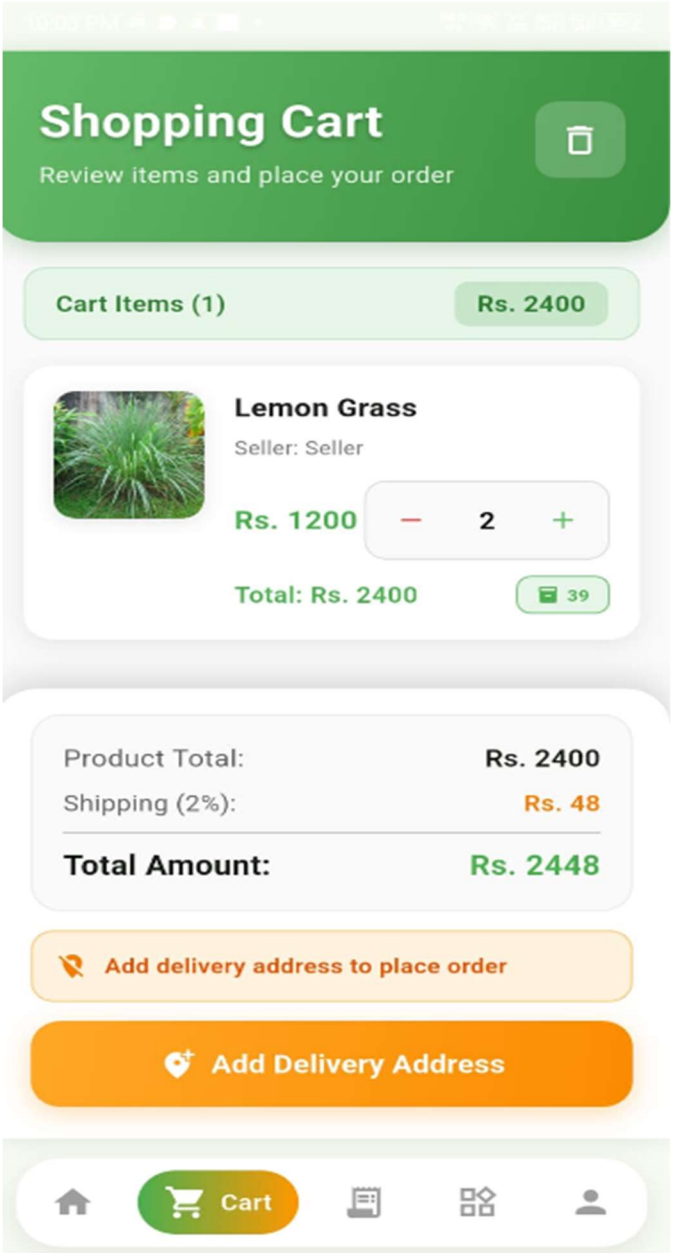

Figure 5.4-4 Shopping Cart

5.4.5 Checkout and Payment Interface

Displays a summary of selected items, shipping address form, payment method selection (Stripe or COD), and total cost, guiding buyers to finalize their purchase. The Stripe payment interface securely collects card details through Stripe's PCI-compliant SDK. For COD orders, the buyer simply confirms the order. Both methods show order confirmation with a success message and order tracking number upon completion.

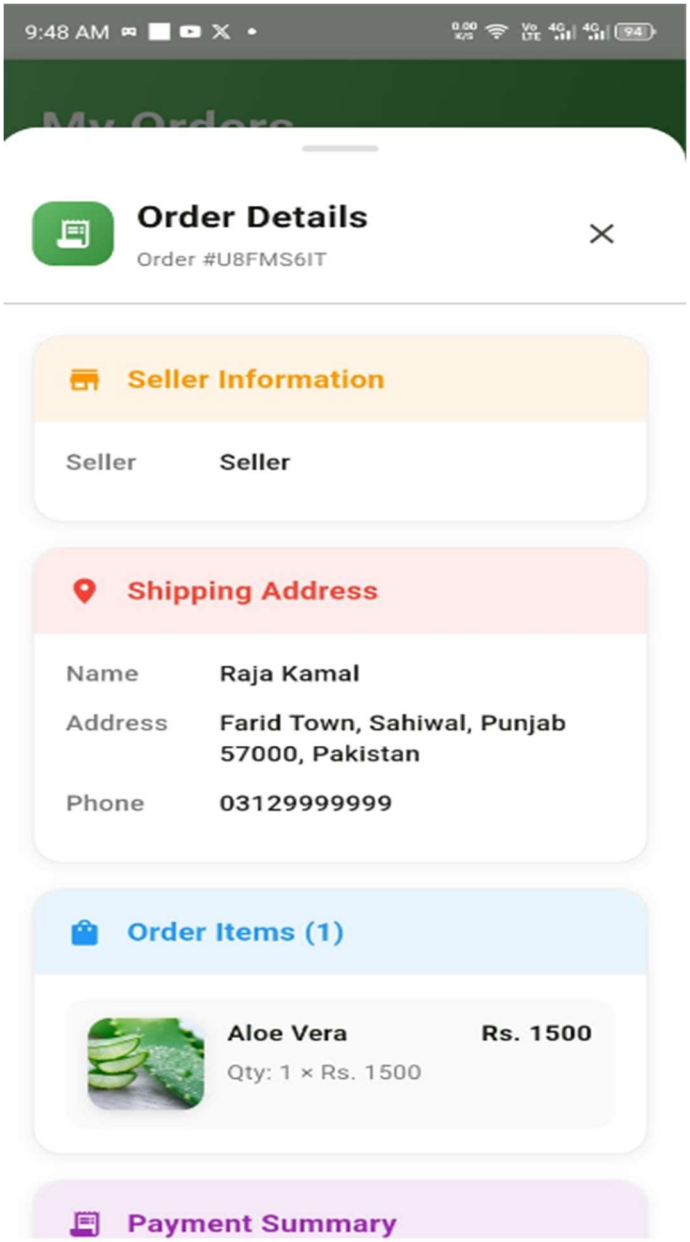

Figure 65.4-5 Payment Success Screen

5.4.6 Order Tracking Screen

Provides buyers with real-time updates on their order progress. The order tracking screen displays the current order status with visual indicators for each stage: Placed, Processing, Shipped, and Delivered. Completed stages are highlighted, and the current stage is animated. Buyers can view their complete order history with details for each past purchase. Each order shows the items purchased, total amount, payment method, and current status.

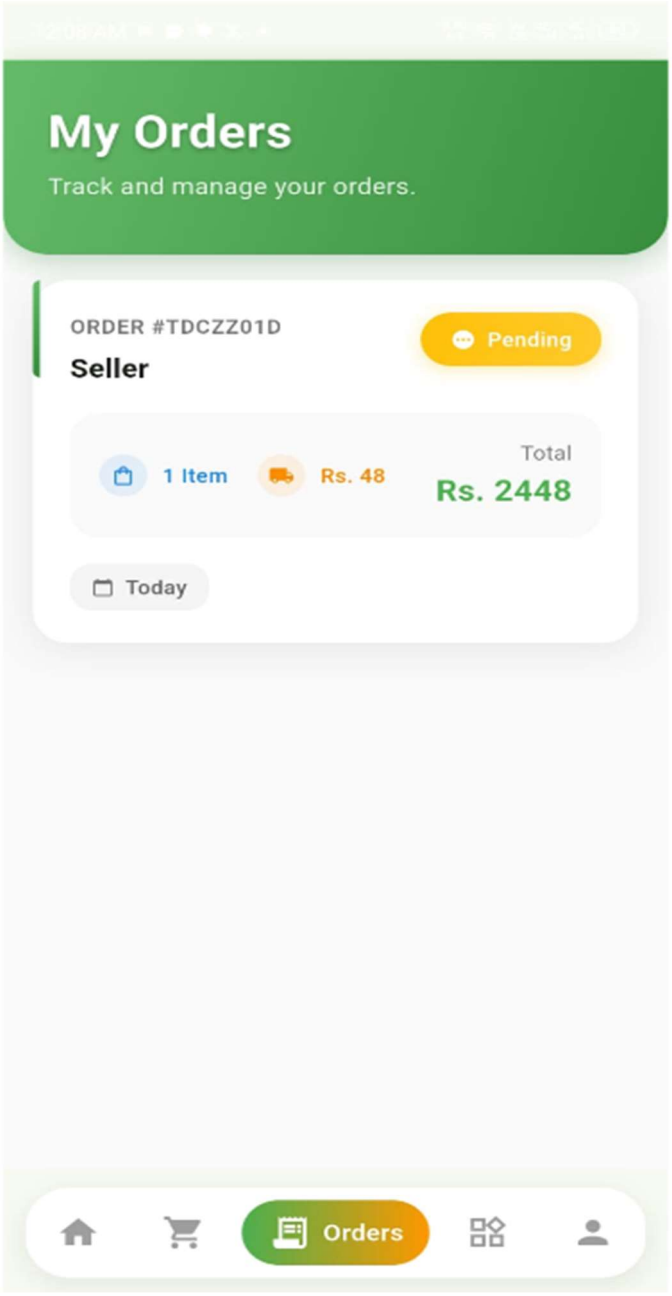

Figure 5.4-6 Order Tracking Screen

5.4.7 Review and Rating Interface

Enables buyers to submit feedback on purchased products after successful delivery. The review screen presents a star rating selector (1-5 stars) and a text input field for written comments. Buyers can read existing reviews from other customers displayed with usernames, ratings, dates, and comment text. This feature helps other buyers make informed purchasing decisions and builds community trust.

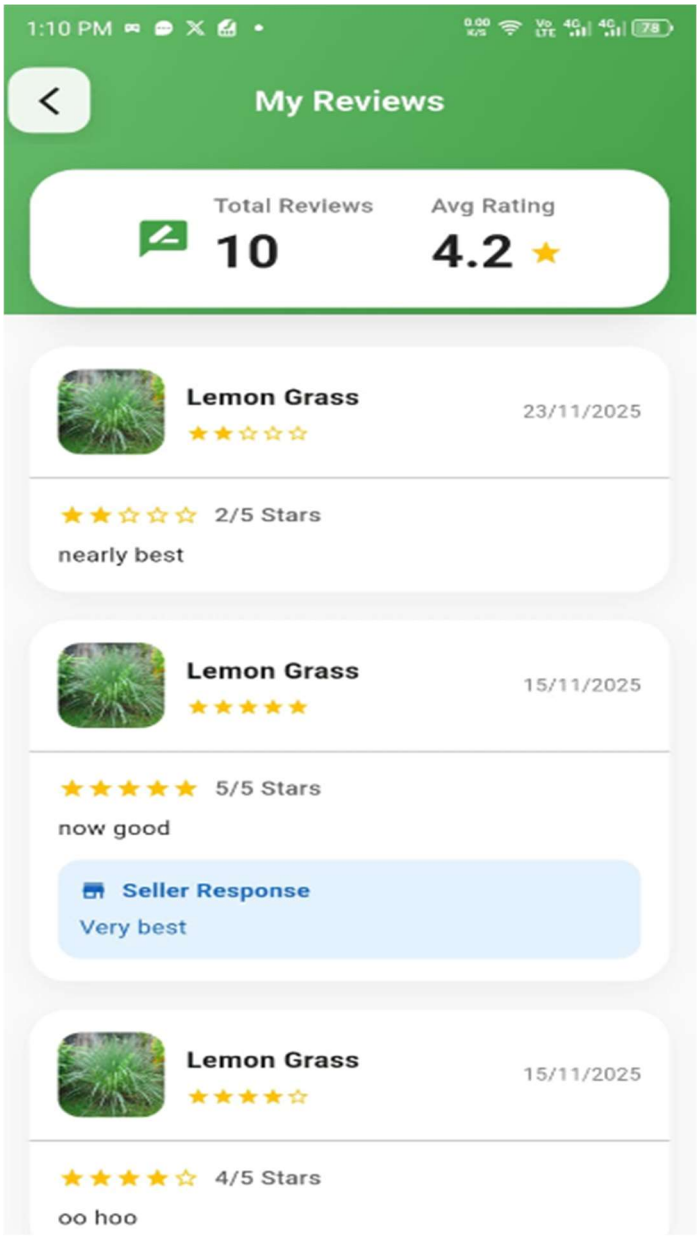

Figure 5.4-7 Review Screen

5.4.8 Seller Dashboard

A dedicated dashboard accessible to sellers for managing their product listings, viewing incoming orders, tracking sales analytics, and monitoring their business performance. The dashboard provides an overview of key metrics including total products listed, active orders, completed sales, and total revenue. Quick action buttons allow sellers to add new products, manage existing inventory, and view order statuses.

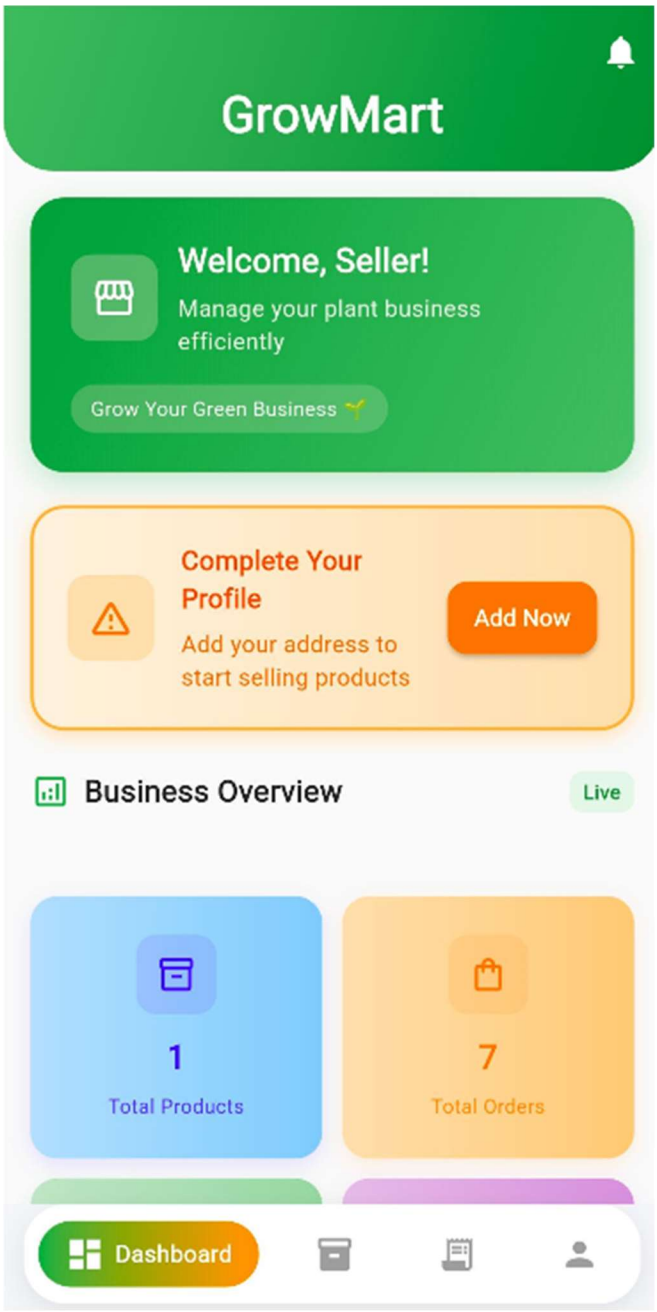

Figure 5.4-8 Seller Dashboard

5.4.9 Seller Product Management

Enables sellers to upload new plant products by entering details such as product title, description, price, category, stock quantity, and uploading corresponding images from their device gallery or camera. Sellers can also edit existing product information, update stock levels, or remove discontinued items from their catalog. The product list view shows all seller products with their current status and sales performance.

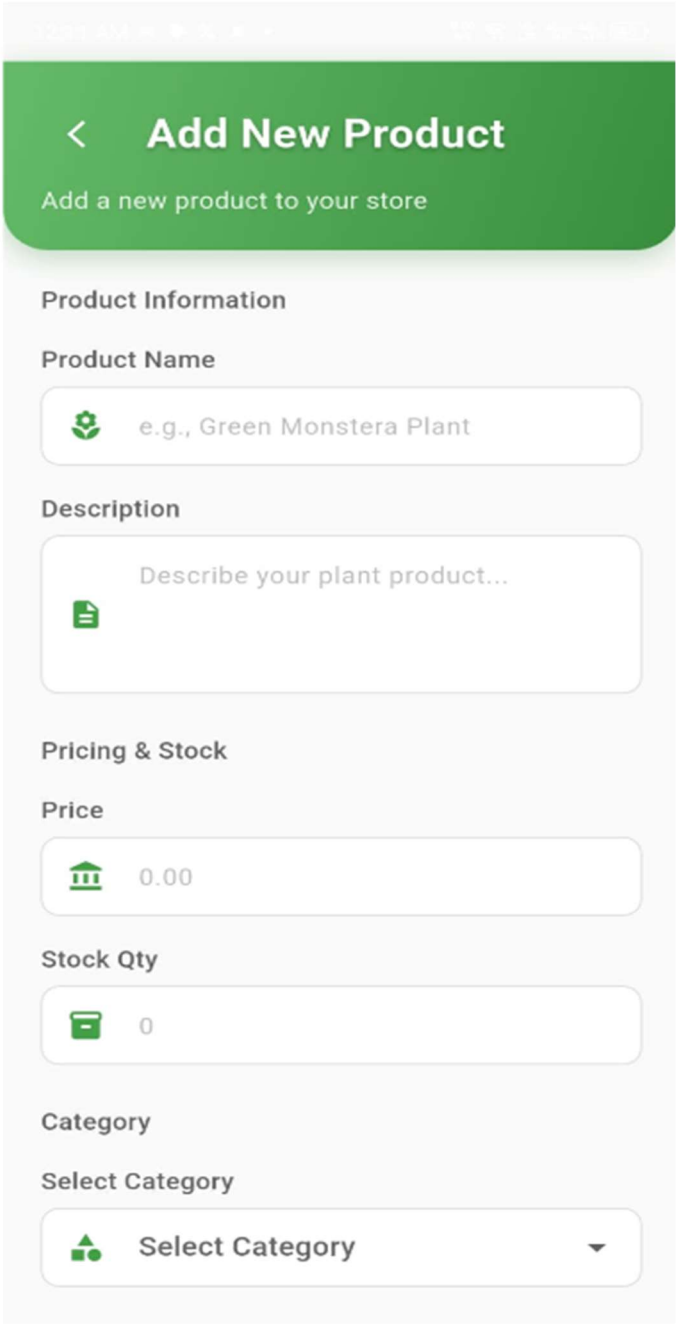

Figure 5.4-9 Seller Add Product Screen

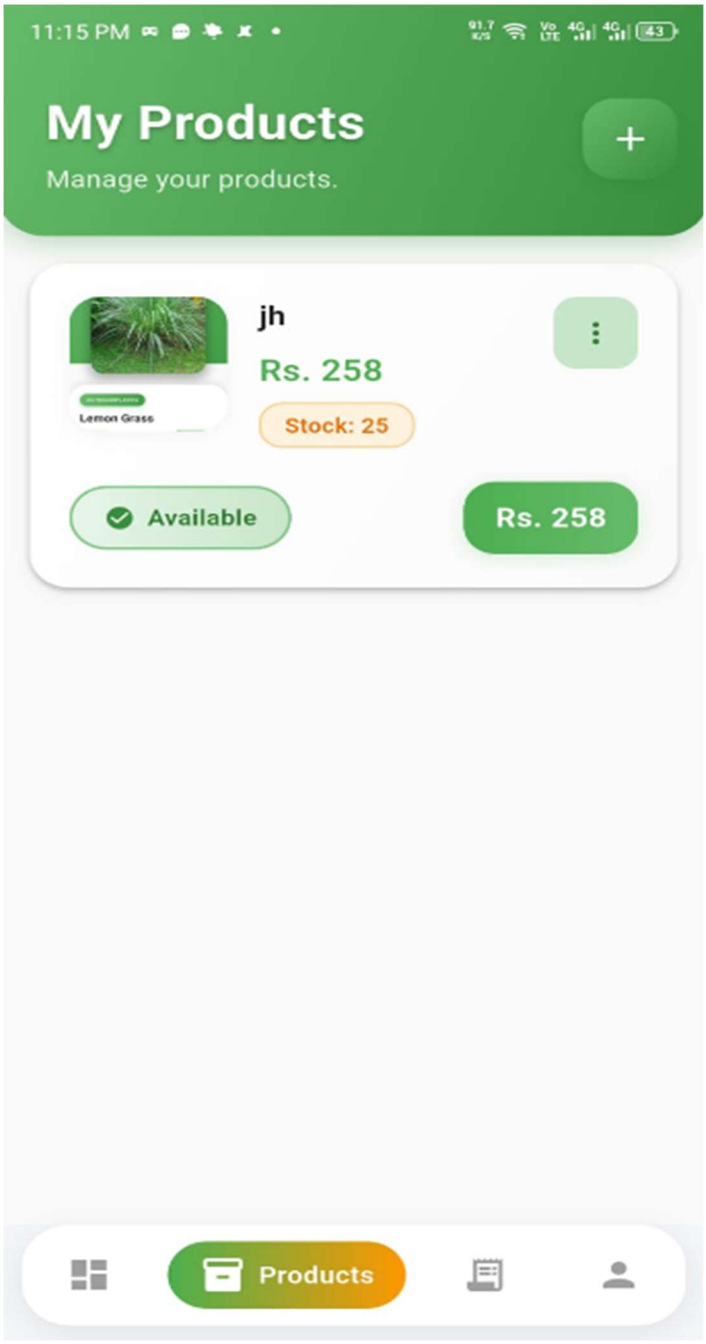

Figure 5.4-10 Seller All Products Screen

5.4.10 Seller Order Management

Displays a list of all customer orders for the seller with relevant details such as buyer name, order ID, products ordered, quantity, total amount, payment status, and order date. The seller can view order details, update order status through the lifecycle stages, and track which orders need attention. Color-coded status indicators help sellers quickly identify pending, processing, and shipped orders.

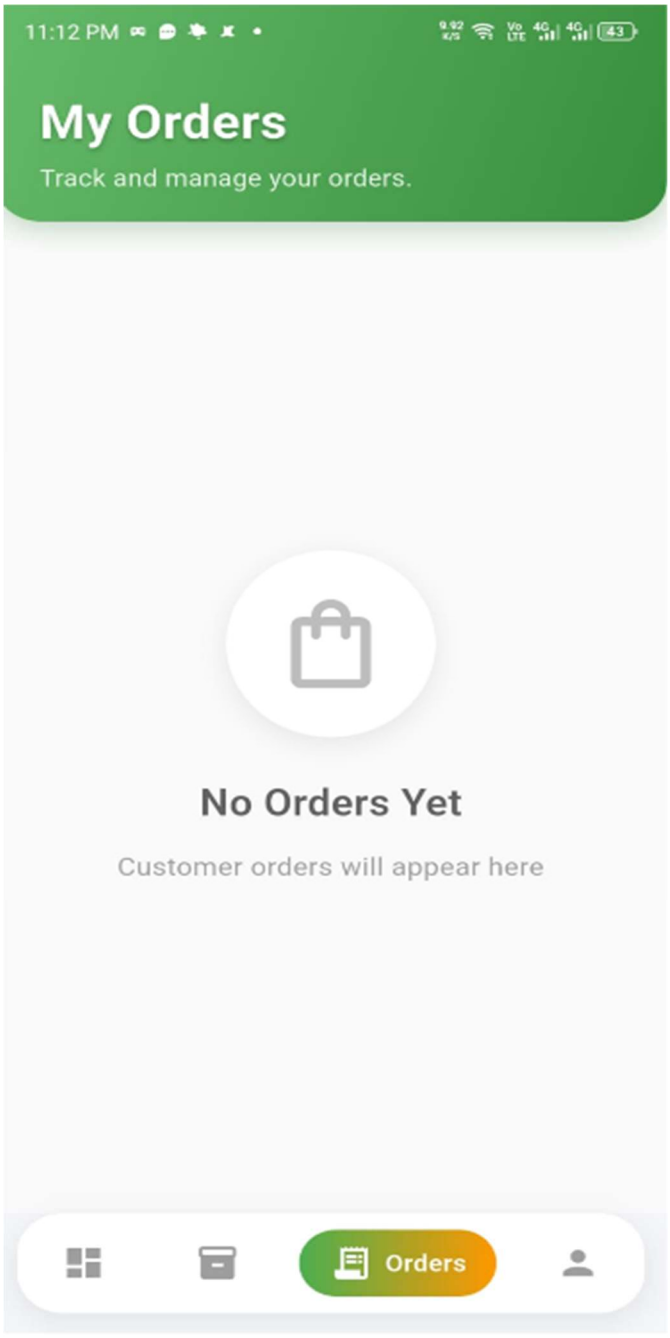

Figure 5.4-11 Seller Order Management Screen

5.4.11 Seller Analytics Dashboard

Provides sellers with visual analytics including charts and graphs showing sales performance, revenue trends over time, most popular products, and order statistics. The analytics dashboard uses MPAndroidChart library to render bar charts, line graphs, and pie charts. This empowers sellers to make data-driven decisions about their inventory, pricing strategies, and business growth.

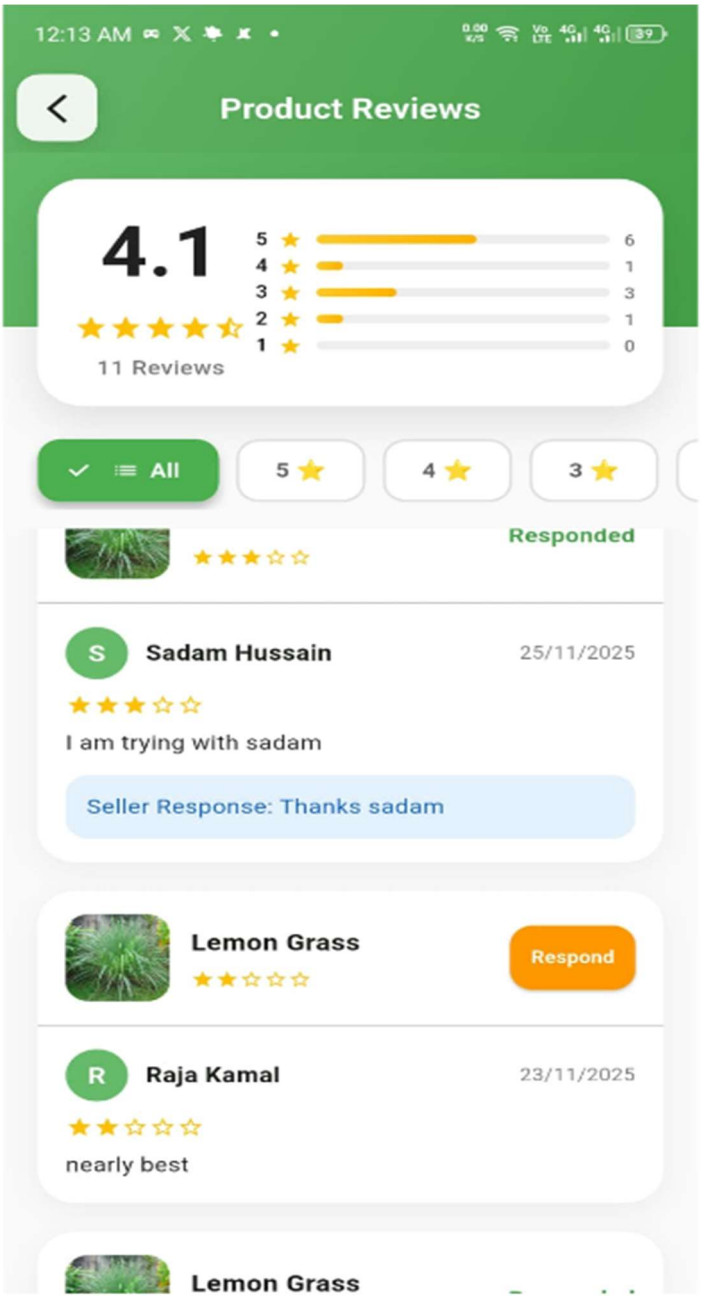

Figure 5.4-12 Seller Analytics Screen

Chapter 6

6.1 Testing and Evaluation

Testing is a vital component in the development lifecycle of the Grow Mart platform, ensuring that the system functions reliably, securely, and efficiently across all user interactions. Given the platform's role-based functionality, real-time data synchronization, integrated payment processing, and push notification delivery, a rigorous testing process is essential to deliver a seamless and error-free user experience. Testing is conducted at various stages of development—from unit tests during feature implementation to comprehensive integration and user acceptance testing before deployment.

The primary goal of testing in Grow Mart is to validate the accuracy of the product management system, verify the order processing logic, ensure the stability of critical modules such as user authentication, cart and checkout workflows, and notification delivery, and confirm that payment processing through Stripe and COD functions correctly. Special attention is also given to testing the Firebase Firestore synchronization to maintain data consistency across all users and the FCM notification delivery to ensure timely communication.

Following best practices in software engineering, testing in Grow Mart is continuous and iterative. Each module undergoes:

- **Unit Testing** to validate individual functionalities such as login validation, product CRUD operations, cart management, and notification triggers.
- **Integration Testing** to verify that modules such as payment gateway, order processing, and notification delivery interact correctly.
- **User Acceptance Testing (UAT)** to ensure that real users can navigate the system effectively and receive the intended experience.

Security testing is also performed to guard against threats such as unauthorized access, data breaches, or injection attacks. Additionally, performance testing is carried out to ensure that the platform handles multiple concurrent users during peak shopping periods without delays or failures.

By implementing a structured and layered testing strategy, the Grow Mart platform ensures that any critical bugs, UI inconsistencies, or performance issues are identified early and addressed promptly. This approach results in a robust, intuitive, and high-performing platform that instills user confidence and trust in the plant shopping experience.

6.2 Manual Testing

Manual Testing is a crucial aspect of the testing process for the Grow Mart platform. It involves testers manually navigating the system to simulate real-world user interactions and validate that all features perform as expected. During this process, testers explore key functionalities such as user login and registration with role selection, product browsing and search with filters, cart management with quantity controls, checkout with both Stripe and COD payment methods, order tracking with status updates, review submission and rating, and notification delivery via FCM.

6.2.1 System Testing

System Testing for Grow Mart is conducted after the development phase to ensure that all platform components work together as a unified, seamless system. This critical stage involves evaluating the integration of core features, including user authentication with role selection, product catalog navigation, cart and checkout flow, payment processing via Stripe and COD, order management and status tracking, review and rating system, and notification delivery through FCM.

The objective of this phase is to confirm that Grow Mart meets all specified requirements in terms of performance, security, data accuracy, and user experience. System testing focuses on identifying potential issues that may arise from the interaction between modules—for example, synchronization failures between cart and product inventory, errors in the checkout or payment workflow, notification delivery delays, or incorrect role-based access control.

By validating the complete end-to-end functionality in a real-world scenario, system testing ensures that the Grow Mart platform delivers a stable, secure, and reliable environment. This guarantees users a dependable experience when browsing, purchasing, and selling plant-related products before the application is released for public use.

6.2.2 Unit Testing

Unit Testing in the Grow Mart platform focuses on verifying the functionality of individual components in isolation to ensure they perform as intended. This includes testing discrete units such as the user login and registration module with Firebase Authentication, product search and filtering logic, cart addition and quantity management, Stripe payment intent creation, order status transitions, review submission and validation, and FCM notification triggers.

For instance, unit tests are applied to functions that handle Firebase Authentication login validation, product CRUD operations in Firestore, cart total calculations, Stripe payment processing, and notification delivery to specific user devices. These tests check whether each component correctly handles inputs (such as login credentials, product details, or payment amounts), processes data accurately, and returns the expected outputs without reliance on other modules.

Developers implement unit tests for all critical backend services and frontend interactions to ensure robustness before integration. This proactive testing approach helps identify and resolve issues early in the development cycle, enabling smoother integration of features across the Grow Mart platform.

6.3 Unit Testing

6.3.1 Unit Testing 1: User Registration

Testing Objective: Ensure the user registration form functions correctly with Firebase Authentication.

Table 6-1 User Registration

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-001	User Registration by entering valid credentials	Name: Test User, Email: testuser@growmart.com, Password: securePass@123, Role: Buyer	User successfully registered in Firebase Auth and Firestore, redirected to Buyer dashboard	Pass
UT-002	User Registration with existing email	Email: existing@growmart.com (already registered)	System displays "User Already Exists" error message	Pass
UT-003	User Registration with missing fields	Name: empty, Email: test@test.com, Password: pass123	System displays validation error for required fields	Pass

6.3.2 Unit Testing 2: Login as User

Testing Objective: Ensure the user login system works correctly for both roles.

Table 6-2 User Login

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-004	Buyer Login with valid credentials	Email: buyer@growmart.com, Password: buyerPass@123	Buyer successfully logs in and sees Buyer	Pass

			dashboard with product catalog	
UT-005	Seller Login with valid credentials	Email: seller@growmart.com, Password: sellerPass@123	Seller successfully logs in and sees Seller dashboard with product management	Pass

6.3.3 Unit Testing 3: Logout as User

Testing Objective: Ensure users can log out successfully from both roles.
Table 6-3 User Logout

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-007	Buyer clicks logout button	Logout action from Buyer dashboard	Buyer successfully exits the system, redirected to login screen	Pass
UT-008	Seller clicks logout button	Logout action from Seller dashboard	Seller successfully exits the system, redirected to login screen	Pass

6.3.4 Unit Testing 4: Add Product to Cart

Testing Objective: Ensure the add-to-cart functionality works correctly.
Table 6-4 Add to Cart

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-009	Add single product to empty cart	Product ID: PL-001, Quantity: 1	Item successfully added to cart with correct quantity, cart total updated	Pass

UT-010	Add multiple quantity of product	Product ID: PL-002, Quantity: 3	Cart shows correct quantity (3) and calculated subtotal	Pass
UT-011	Add product exceeding available stock	Product ID: PL-003, Quantity: 50 (stock: 10)	System displays "Insufficient stock" error message	Pass

Testing Objective: Ensure seller can add a new product listing with image upload.
Table 6-5 Seller Add Product

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
UT-012	Seller adds new product with image	Title: "Snake Plant", Price: 1500, Category: Indoor, Stock: 10, Image: plant.jpg	Product successfully added to Firestore, image uploaded to Firebase Storage, visible in catalog	Pass
UT-013	Seller adds product without image	Title: "Rose Plant", Price: 800, Category: Outdoor, Stock: 5, Image: none	System displays validation error "Product image required"	Pass
UT-014	Seller adds product with missing title	Title: empty, Price: 500, Category: Tools	System displays validation error for required fields	Pass

6.4 Functional Testing

6.4.1 Functional Testing 1: Login with Different Roles

Objective: Ensure role-based access control works correctly and appropriate dashboards load.
Table 3-6 Login as Different Roles

No.	Test Case	Attribute & Value	Expected Result	Result
1	Login as Buyer	Email: buyer@growmart.com,	Buyer	Pass

		Password: buyerPass@123	dashboard loads with product catalog, cart, and order tracking options	
2	Login as Seller	Email: seller@growmart.com, Password: sellerPass@123	Seller dashboard loads with product management, order management, and analytics options	Pass
3	Login as Unregistered User	Email: unknown@growmart.com, Password: test123	System displays "No account found" error message	Pass

6.4.2 Functional Testing 2: Browse Products with Different Categories

Objective: Ensure category filtering and search functionality works correctly.

Table 6-7 Browse Products by Category

No.	Test Case	Attribute & Value	Expected Result	Result
1	Browse Indoor Plants	Category: Indoor	Only indoor plant products displayed	Pass
2	Browse Gardening Tools	Category: Tools	Only gardening tool products displayed	Pass
3	Search by Keyword	Search: "Fertilizer"	Products matching "Fertilizer" in title or description displayed	Pass
4	Filter by Price	Price: 500 to	Only products	Pass

	Range	2000	within specified price range displayed	
5	No Results Search	Search: "xyzabc123"	"No Results Found" message displayed	Pass

6.4.3 Functional Testing 3: Payment Processing

Objective: Ensure both Stripe and COD payment methods work correctly.
Table 6-8 Payment Methods

No.	Test Case	Attribute & Value	Expected Result	Result
1	Stripe Payment with valid card	Card: 4242 4242 4242 4242, Exp: 12/28, CVV: 123, Amount: 2500	Payment processed successfully, order status PAID, order created in Firestore	Pass
2	Cash on Delivery	COD selected, Amount: 1800	Order confirmed with PENDING payment status, order created in Firestore	Pass
3	Stripe Payment with invalid card	Card: 4000 0000 0000 0002 (declined)	Payment declined, error message displayed, cart preserved for retry	Pass
4	Stripe Payment network failure	Simulate network disconnect during payment	System displays "Network error, please try again" message	Pass

6.5 Integration Testing

Objective: Ensure modules such as login, product browsing, cart, checkout, payment, order management, and notifications work cohesively as an integrated system.

Table 6-9 Integration Testing

No.	Test Case	Attribute & Value	Expected Result	Result
1	Login and Browse Products	Email: buyer@growmart.com → Browse categories	Buyer successfully logs in and views categorized product catalog from Firestore	Pass
2	Add to Cart and Checkout	Select product → Add to cart → Proceed to checkout	Cart updates correctly, checkout screen displays correct items with calculated total	Pass
3	Complete Order Flow (Stripe)	Checkout → Stripe payment → Order created → Inventory updated	Payment processed, order created in Firestore, stock reduced, buyer notified	Pass
4	Seller Receives Order Notification	Buyer places order → Seller FCM notification	Seller receives push notification with order details within 5 seconds	Pass
5	Seller Updates Order Status	Seller changes status to PROCESSING → SHIPPED	Order status updated in Firestore, buyer receives	Pass

			FCM notification for each status change	
6	Delivery and Review Flow	Order marked DELIVERED → Buyer submits review	Review saved to Firestore, displayed on product page, seller receives FCM notification	Pass
7	Contact Support Form	Name: John Doe, Email: john@test.com, Message: "Issue with order #123"	Message stored in Firestore support collection, support notification sent to admin	Pass
8	Seller Analytics Update	Multiple orders placed → Seller views analytics dashboard	Charts updated with correct sales data, revenue totals, and order statistics	Pass

Chapter 7

7.1 Conclusion

The Grow Mart platform offers an advanced and interactive solution for enhancing the plant and gardening e-commerce experience through specialized mobile technology. Designed and developed using Flutter and Firebase, the system integrates essential e-commerce functionalities, including user authentication with role selection, product management, shopping cart, secure checkout via Stripe and COD, order tracking, review and rating systems, real-time notifications, and seller analytics.

At the core of the platform lies a robust role-based architecture that separates buyer and seller functionalities, providing each user type with a tailored experience. Buyers can browse categorized plant products, search with smart filters, manage their cart, complete purchases through secure payments, track orders in real-time, and leave reviews. Sellers can upload and manage product listings, process incoming orders, update delivery statuses, and monitor their business performance through visual analytics dashboards.

The system utilizes Firebase Firestore for real-time data synchronization, Firebase Authentication for secure user management, Firebase Storage for product image hosting, and Firebase Cloud Messaging for instant push notifications. Stripe SDK integration ensures PCI-compliant payment processing, while the MVVM architecture with Flutter ensures a responsive, maintainable, and scalable codebase.

Unlike many general marketplace platforms that lack plant-specific focus, Grow Mart is developed as a specialized Flutter application, ensuring complete control over both frontend and backend logic. The use of Firebase Firestore for structured data storage and Firebase Storage for optimized image management supports smooth performance and scalability.

The simple and responsive interface makes navigation effortless for users while maintaining flexibility for sellers managing their inventory and orders. By combining traditional e-commerce components with modern mobile development technologies, Grow Mart demonstrates a practical and innovative approach to solving real-world challenges in niche e-commerce.

The project effectively bridges the gap between general online marketplaces and specialized plant commerce, offering a reliable, accessible, and user-centric platform that caters to both plant buyers and sellers.

7.2 Future Work

While the current version of Grow Mart successfully delivers a seamless plant buying and selling experience, several opportunities exist to further elevate the platform's functionality, scalability, and user engagement. Future work can focus on the following areas:

7.2.1 AI-Based Plant Recognition

Integrating artificial intelligence and machine learning models to enable plant recognition through the device camera. Users could take a photo of a plant, and the system would identify the species, provide care information, and suggest similar products available on the platform. Using deep learning algorithms for image classification, this feature would significantly enhance user engagement and provide added value to plant enthusiasts. The system could also detect plant diseases from leaf images and recommend appropriate fertilizers or treatments available on the marketplace.

7.2.2 GPS-Based Delivery Tracking

Implementing real-time GPS tracking for orders, allowing buyers to monitor the exact location of their delivery in transit. This would provide greater transparency and confidence in the delivery process, similar to modern food delivery applications. Sellers and delivery personnel could update location data in real-time, and buyers would see an interactive map showing their order's journey from the seller to their doorstep.

7.2.3 Multi-Platform Support (iOS and Web)

Expanding Grow Mart beyond Android to include iOS and web platforms. Flutter's cross-platform capabilities make this expansion feasible with minimal code modifications, significantly broadening the user base and accessibility. A web-based admin panel would also enable better platform management and oversight.

7.2.4 Multi-Vendor Marketplace Enhancement

Enhancing the multi-seller capabilities to support a broader range of sellers with features like seller verification badges, featured listings, promotional tools, and commission management. This would transform Grow Mart into a fully-fledged plant commerce marketplace where multiple sellers can compete and collaborate, offering users a wider variety of products and competitive pricing.

7.2.5 Augmented Reality (AR) Integration

Implementing AR features that allow users to visualize plants in their actual space before purchasing. Users could see how a potted plant would look in their room or garden, enhancing the shopping experience and reducing returns. Using ARCore and Flutter's AR plugins, buyers could place virtual plants in their environment through their smartphone camera.

7.2.6 Advanced Admin Analytics

Future improvements will include upgrading the admin panel to incorporate advanced visual analytics, enabling administrators to gain deeper insights into platform performance and user engagement. This includes integrating dashboards that display real-time data on user behavior, most-purchased products, peak usage times, conversion rates, and platform revenue.

7.2.7 Accessibility and Localization

To ensure a more inclusive user experience, future development will focus on enhancing accessibility features across the Grow Mart platform. This includes implementing screen reader compatibility, adjustable text sizes, high-contrast mode for visually impaired users, and support for multiple languages. Multi-language support will broaden the platform's usability for a global audience, allowing users to navigate the interface in their preferred language.

7.2.8 Enhanced Payment Gateway Support

Expanding current payment support with integrations for regional payment methods such as Jazzcash, Easypaisa, or direct bank transfers for smoother transactions tailored to local market preferences. This would make the platform more accessible to users who prefer alternative payment methods beyond traditional credit cards. Through continuous iteration and user-centered improvements, Grow Mart can evolve into a robust platform that not only facilitates plant commerce but also redefines how customers interact with gardening and plant care technology.

Chapter 8

8.1 References

[1] Firebase Documentation. (2024). Firebase Services Overview. https://firebase.google.com/docs
[2] Flutter Documentation. (2024). Flutter Framework Guide. https://docs.flutter.dev
[3] Stripe API Documentation. (2024). Stripe Payment Integration. https://stripe.com/docs/api
[4] Android Developer Guide. (2024). Android Development Best Practices. https://developer.android.com/guide
[5] Flutter Samples Repository. (2024). Official Flutter Code Samples. https://github.com/flutter/samples
[6] IEEE Xplore Digital Library. (2023). E-commerce and Mobile Application Development Research. IEEE Transactions on Software Engineering.
[7] Google Material Design 3 Guidelines. (2024). Design System for Mobile Applications. https://m3.material.io
[8] MPAndroidChart Library Documentation. (2024). Chart Visualization for Android. https://github.com/PhilJay/MPAndroidChart
[9] Intext PDF Library Documentation. (2024). PDF Generation for Mobile Applications. https://itextpdf.com/en/resources/api-documentation
[10] Agile Modeling Artifacts. (2024). Software Design and Development Models. http://agilemodeling.com/artifacts/
[11] Rubin, K. S. (2012). Essential Scrum: A Practical Guide to the Most Popular Agile Process. Addison-Wesley Professional.
[12] Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
[13] Google Firebase Security Rules Documentation. (2024). Securing Data in Cloud Firestore. https://firebase.google.com/docs/firestore/security/get-started
[14] McConnell, S. (2004). Code Complete: A Practical Handbook of Software Construction (2nd ed.). Microsoft Press.
[15] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.